

Expert Parallelism in LLM Training

Autumn 2025

Lecturer: Yuedong (Steven) Xu

Shenzhen Loop Area Institute

yuedongxu@slai.edu.cn

Fudan University

ydxu@fudan.edu.cn

Disclaimer

Machine learning systems is a broad and rapidly evolving field. The course material has been developed using a broad spectrum of resources, including research papers, lecture slides, blogposts, research talks, tutorial videos, and other materials shared by the research community.

Distributed LLM Training: Outline

- Data Parallelism
 - Parameter-Server, All-Reduce, Memory Optimization
- Model Parallelism
 - Pipeline Parallelism, Tensor Parallelism, Sequence Parallelism
- Mixture of Experts
 - **Principles**
 - Parallel Training
 - DeepEP

Mixture of Experts

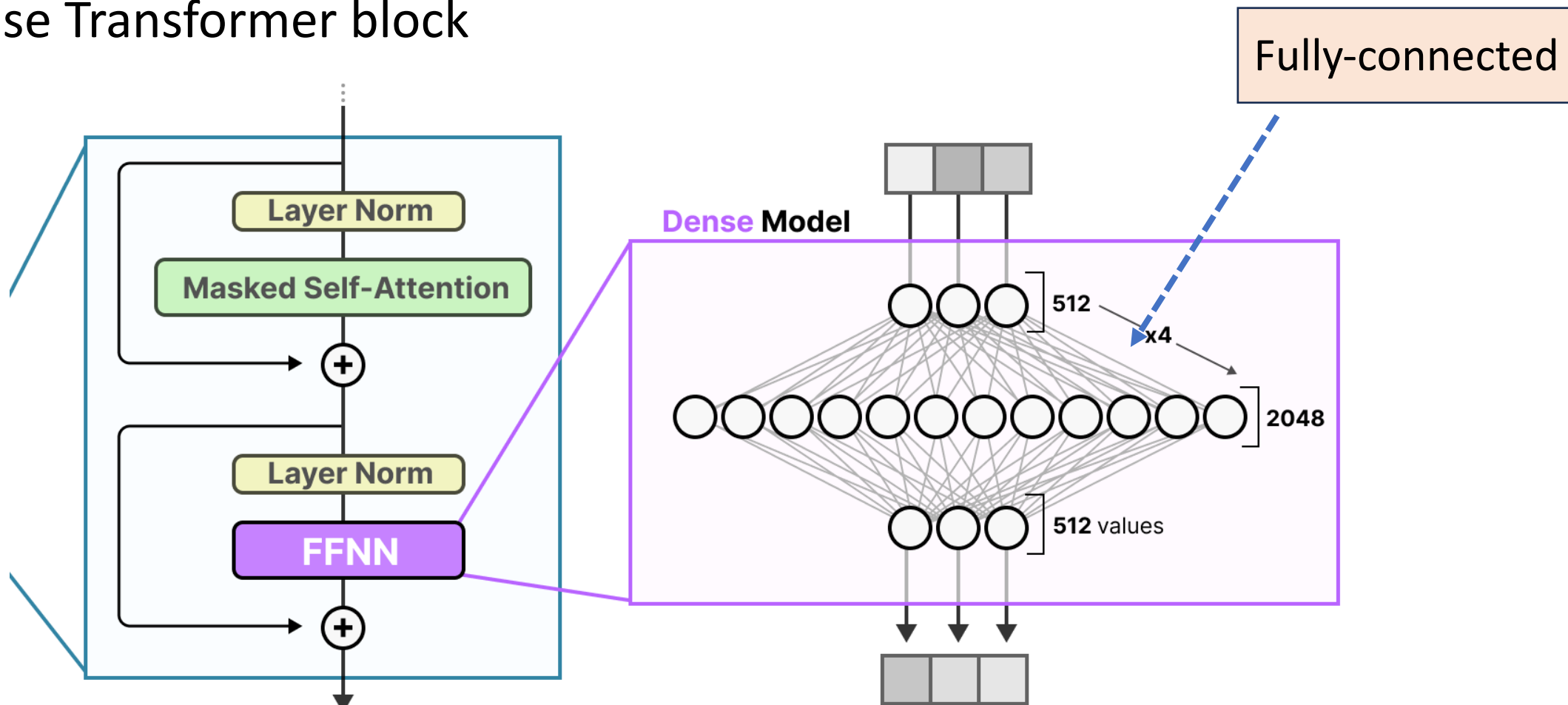
- Mixture of Experts in Early Days
 - Robert Jacobs, Michael Jordan, Steven Nowlan and Geoffrey Hinton (1991)
 - “a new supervised learning procedure for systems composed of many separate networks, each of which learns to handle a subset of the complete set of training cases.”
 - David Eigen, Marc'Aurelio Ranzato, Ilya Sutskever (2013)
 - “extend the Mixture of Experts to a stacked model, the Deep Mixture of Experts, with multiple sets of gating and experts.”
- Mixture of Experts in Transformer
 - GShard by Google

What shall we learn in this lecture

- We only learn
 - the structure of MoE in today's language models
 - the system challenges faced by MoE model training
 - recent progresses on addressing All-to-All issues
- We encourage students to learn by themselves
 - the instability of MoE model training
 - the design of new MoE framework
 - the fine-tuning techniques of MoE models

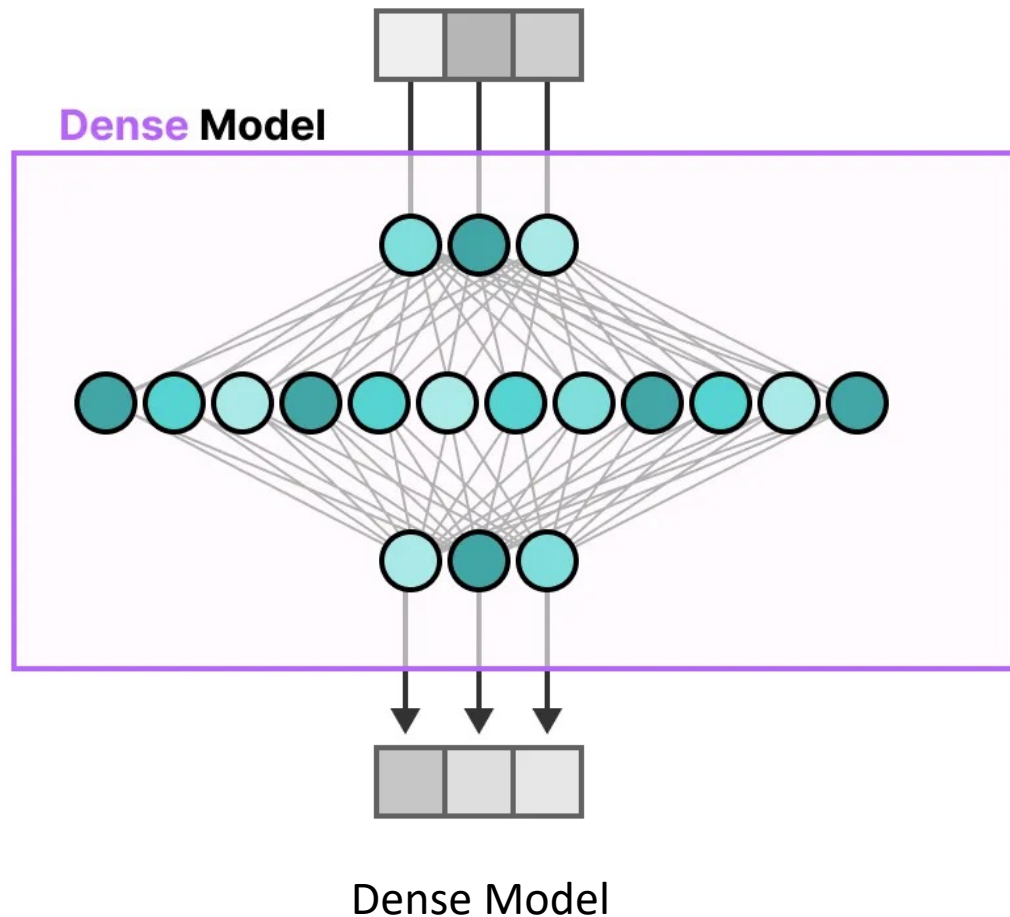
Mixture of Experts

- Dense Transformer block



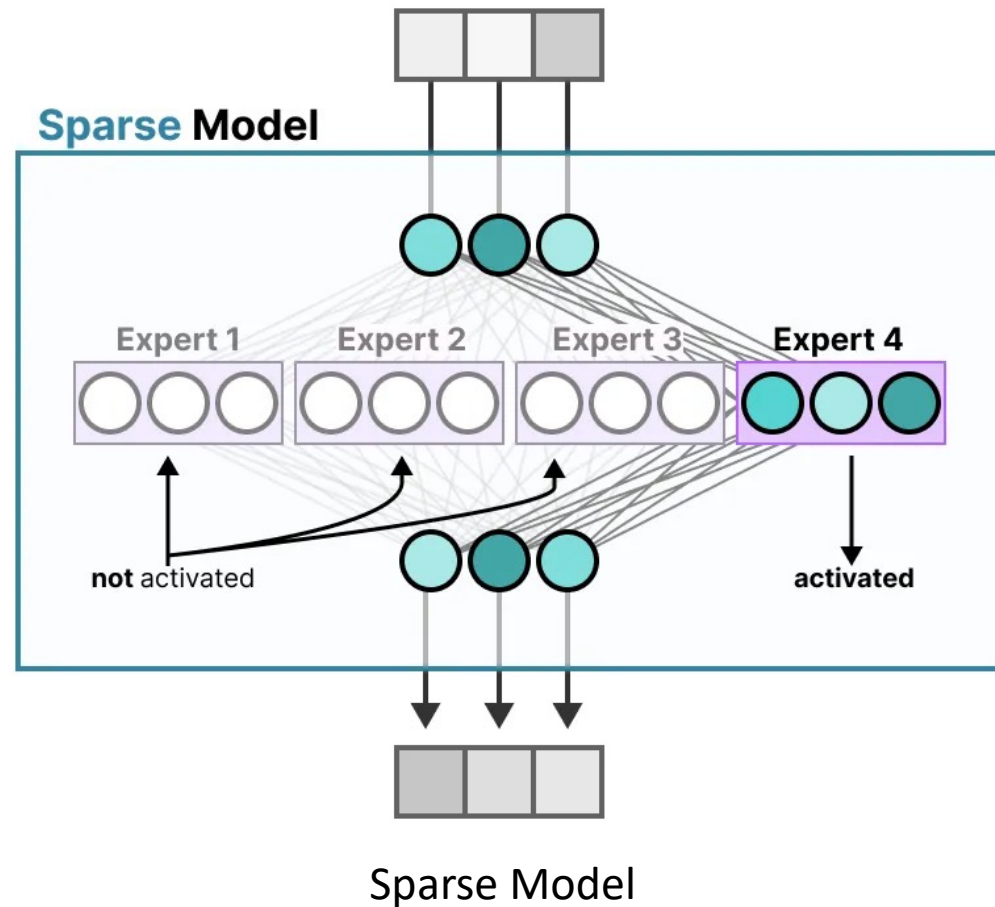
Mixture of Experts

SLAI is an international institute

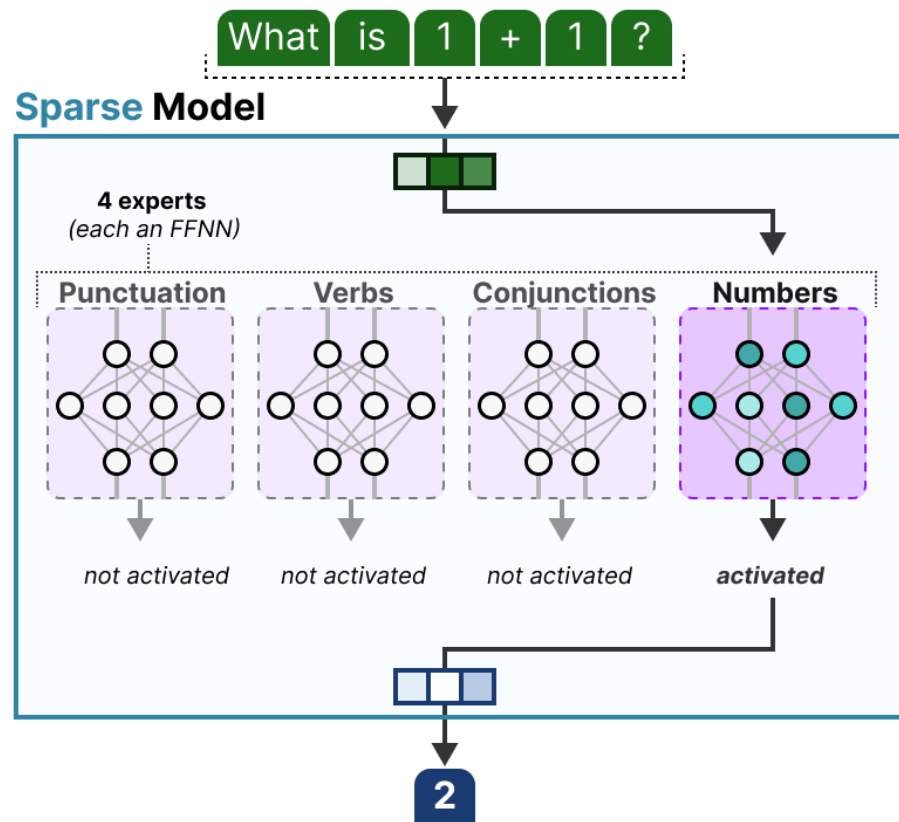


copy

SLAI is an international institute



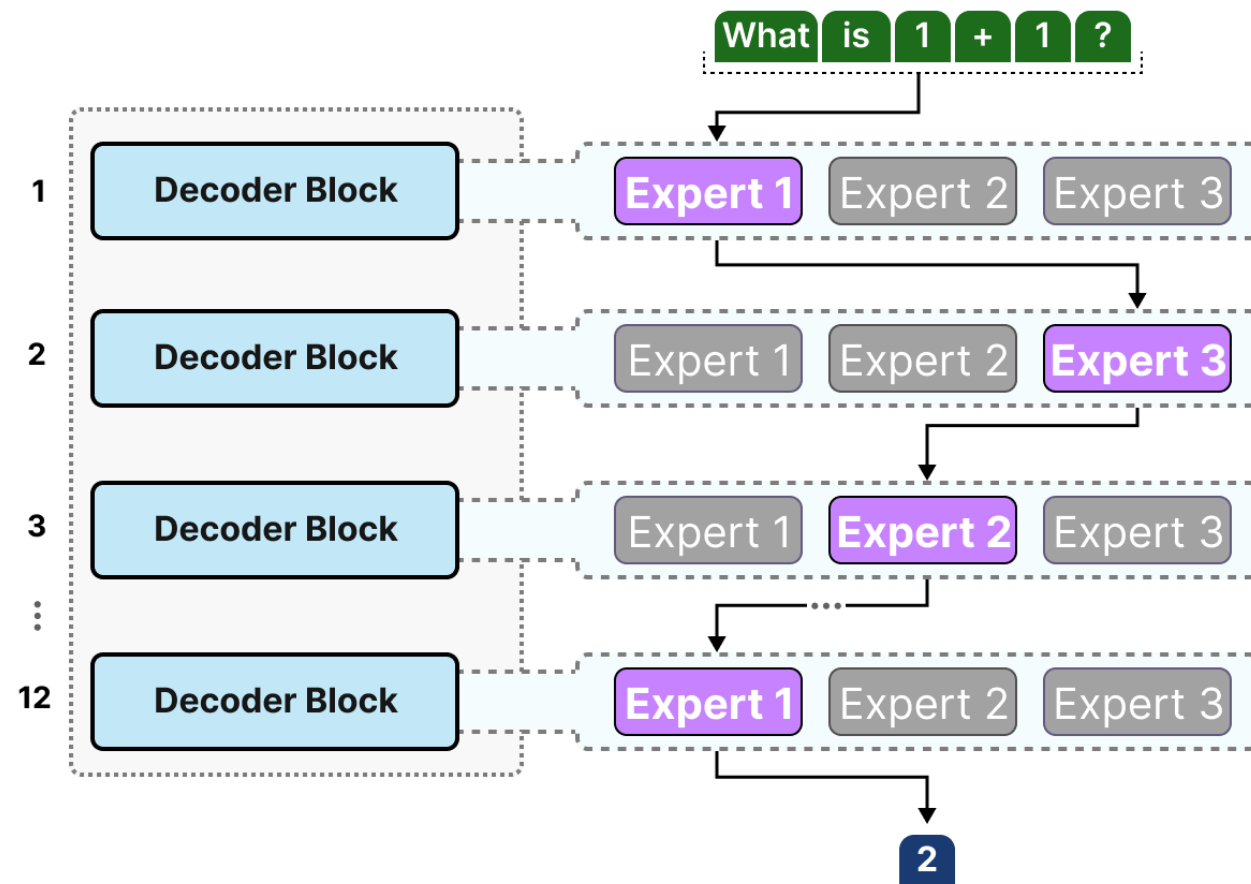
Mixture of Experts



Each expert specializes in different syntactic tokens



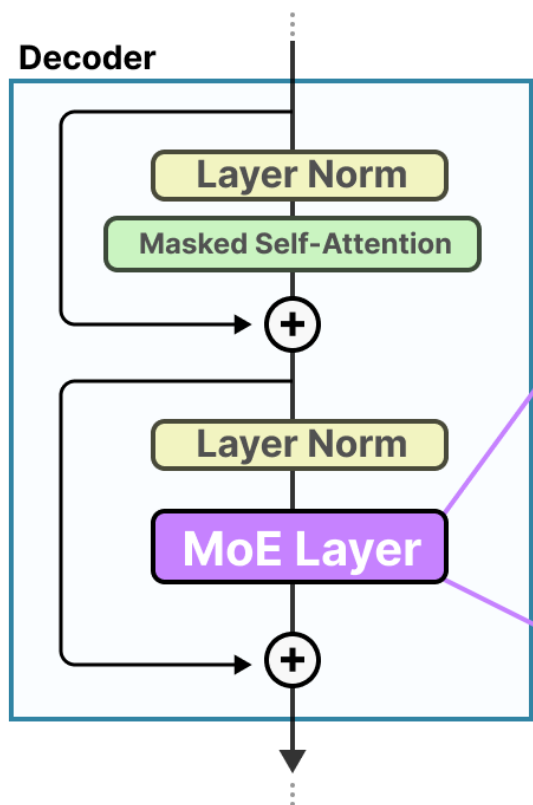
stacking



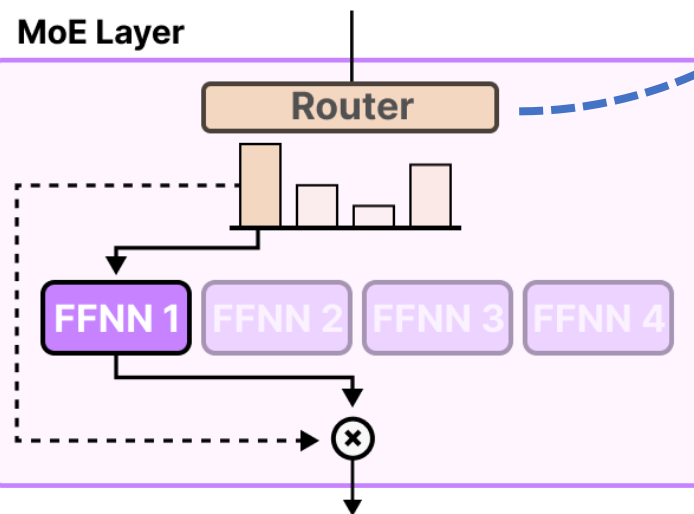
More than one decoders → The chosen experts likely differ between tokens, and each token chooses different experts when traversing different encoders → a path exists

Mixture of Experts

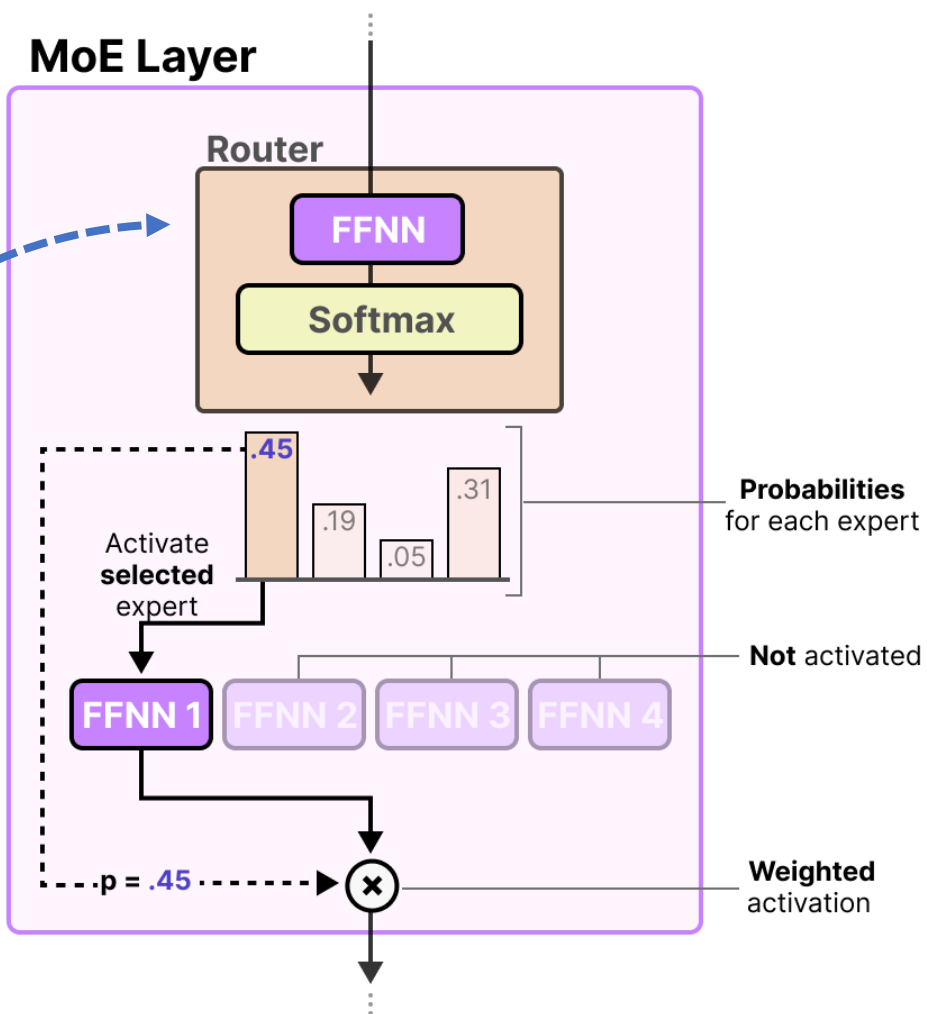
- How to select an expert?



Transformer layer

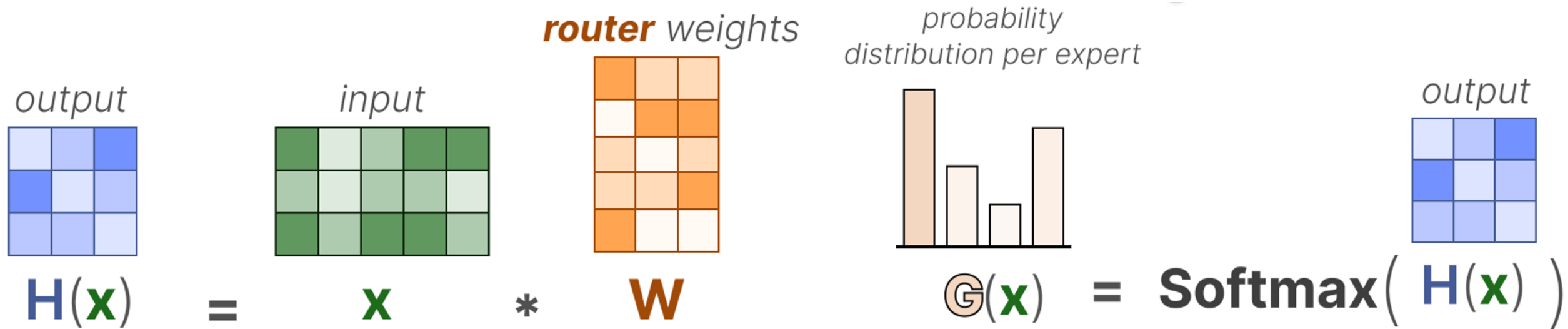


Determines which tokens are sent to which experts



Mixture of Experts

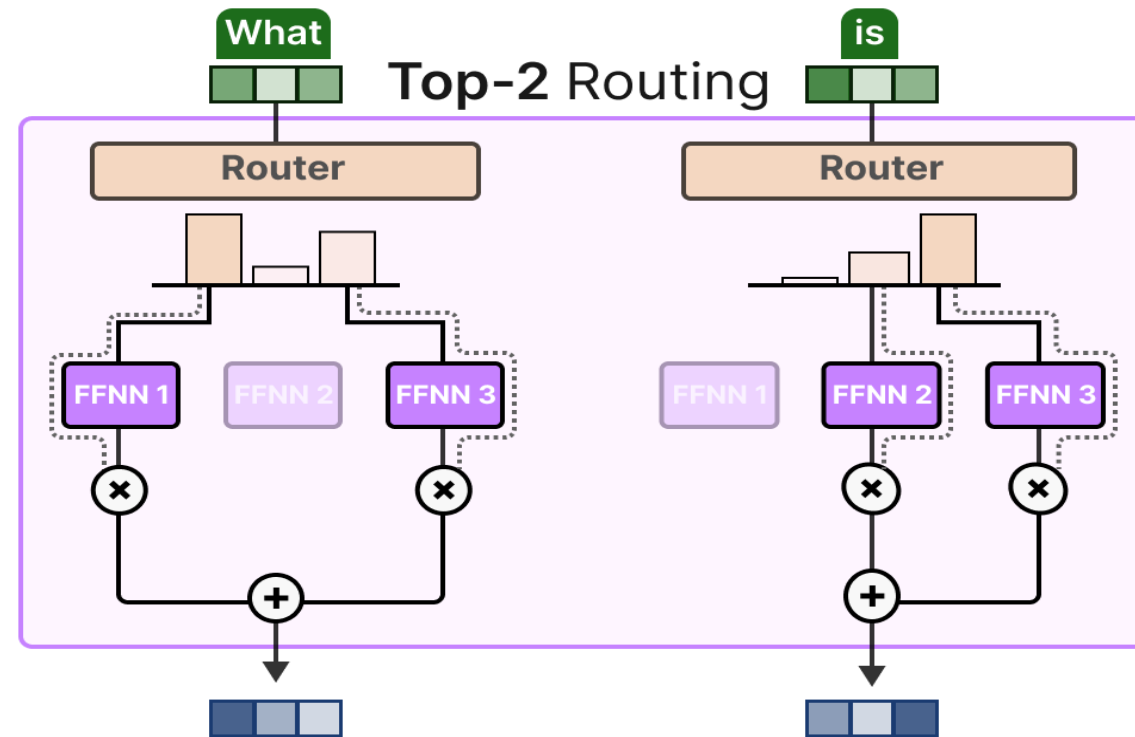
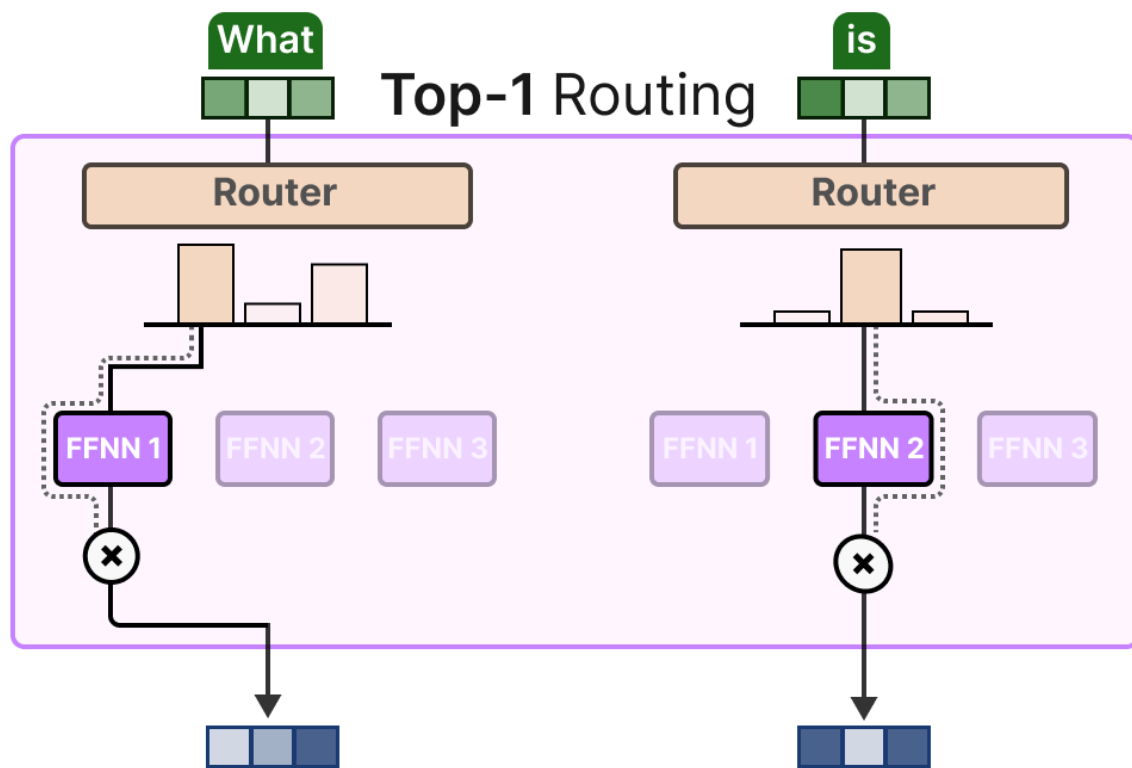
- How to select an expert?



A very basic router

Mixture of Experts

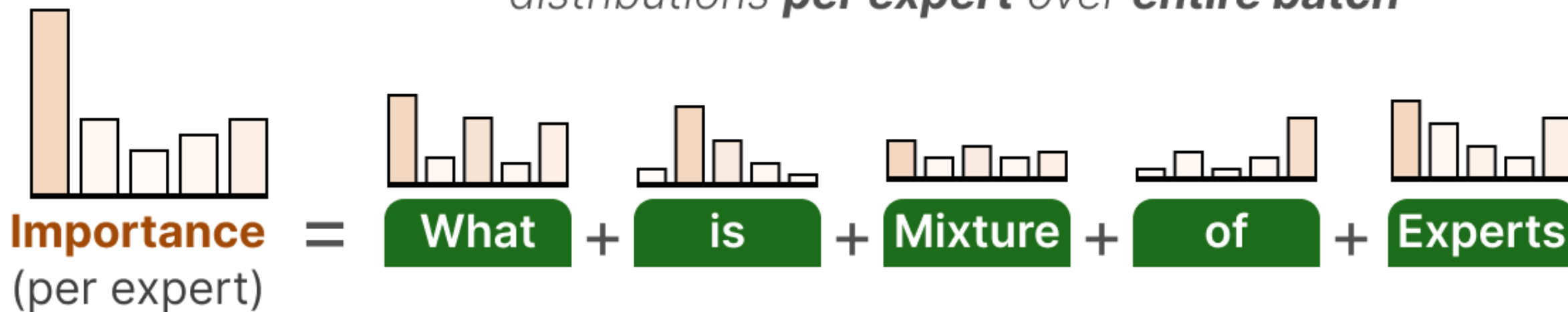
- Top-K routing



Mixture of Experts

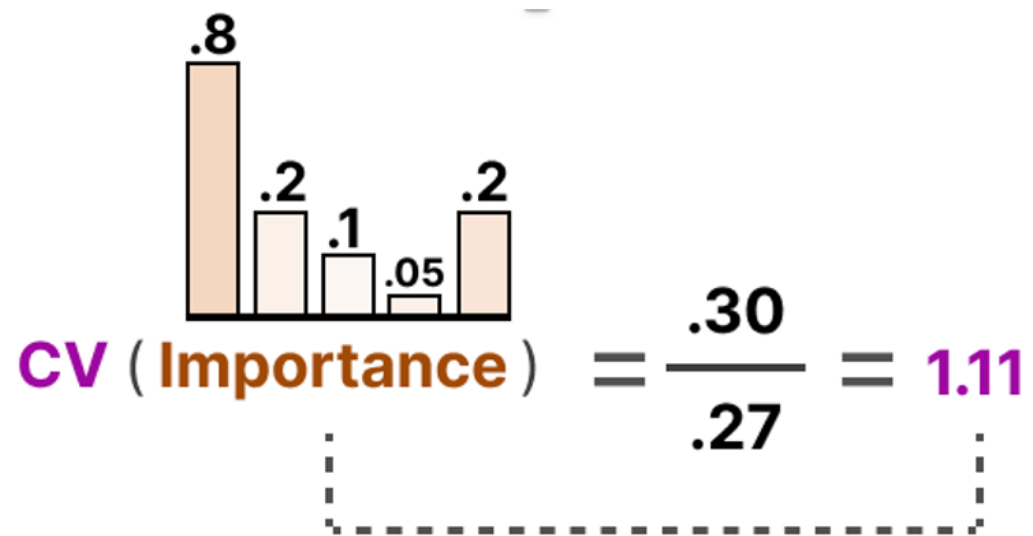
- Load balancing from an algorithmic perspective
 - Quantifying importance of experts

*sum probability
distributions **per expert** over **entire batch***



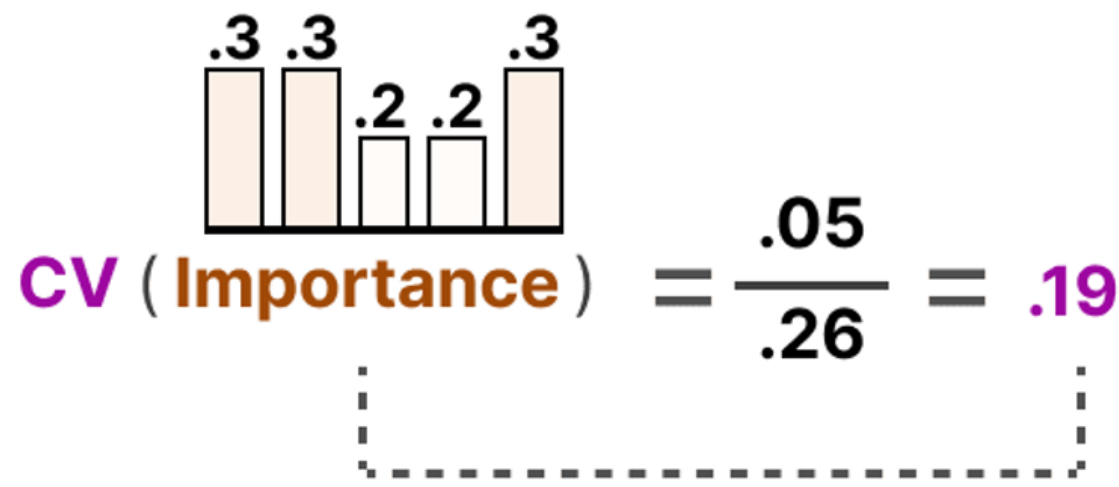
Mixture of Experts

- Load balancing from an algorithmic perspective
 - Coefficient of variation to capture the diversity across experts



High variance in expert
importance result in **high CV**

Unbalanced



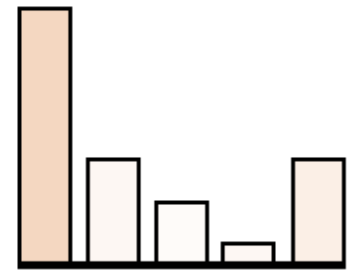
Low variance in expert
importance result in **low CV**

More balanced

Mixture of Experts

- Load balancing from an algorithmic perspective
 - Adding “auxiliary loss” to the overall training objective function

Auxiliary Loss = $\overset{\text{(constant) scaling factor}}{w_{\text{importance}}} * \text{CV}(\text{Importance})^2$

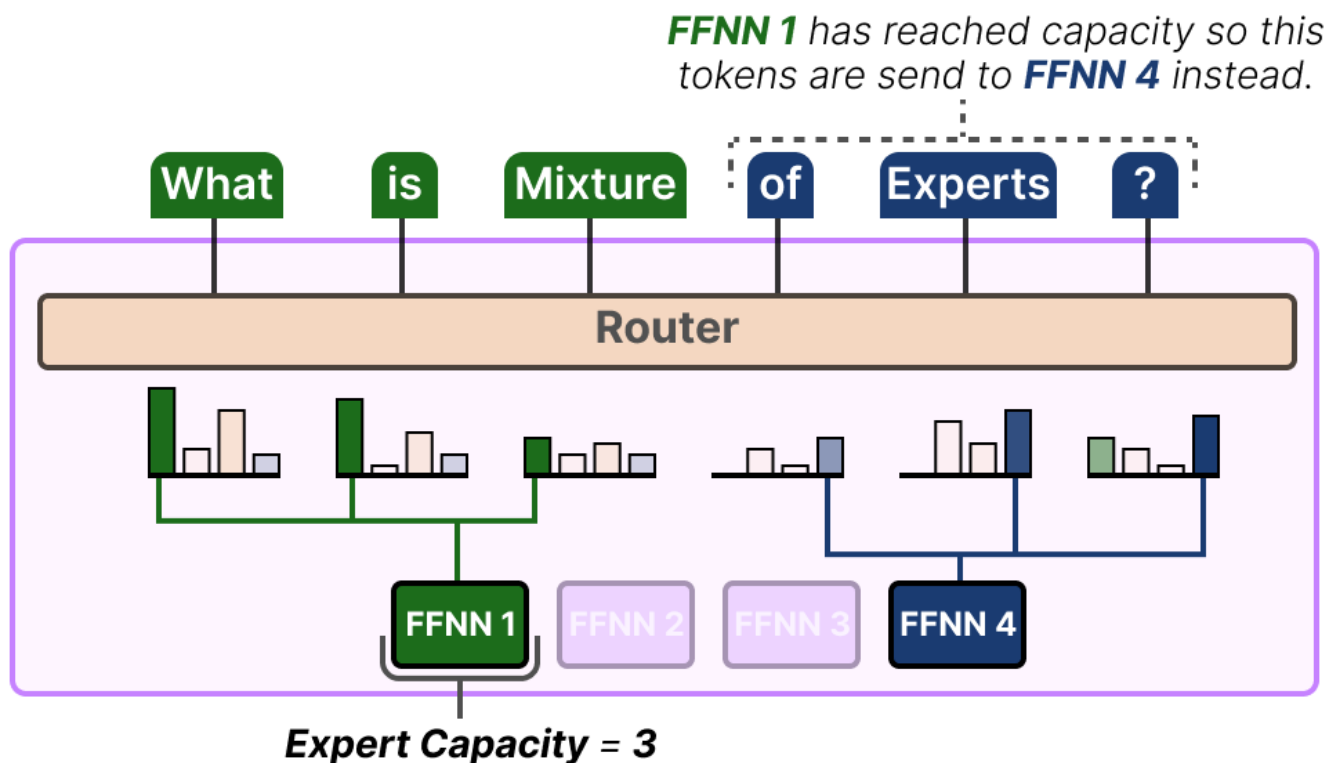


A high variance in expert importance (CV) results in high loss and vice versa

$$l_{aux} = \frac{1}{E} \sum_{e=1}^E \frac{c_e}{S} * m_e$$

Mixture of Experts

- Expert Capacity
 - limiting the amount of tokens a given expert can handle



$$capacity = \max\left(\frac{S}{E} * K * capacity_factor, min_capacity\right)$$

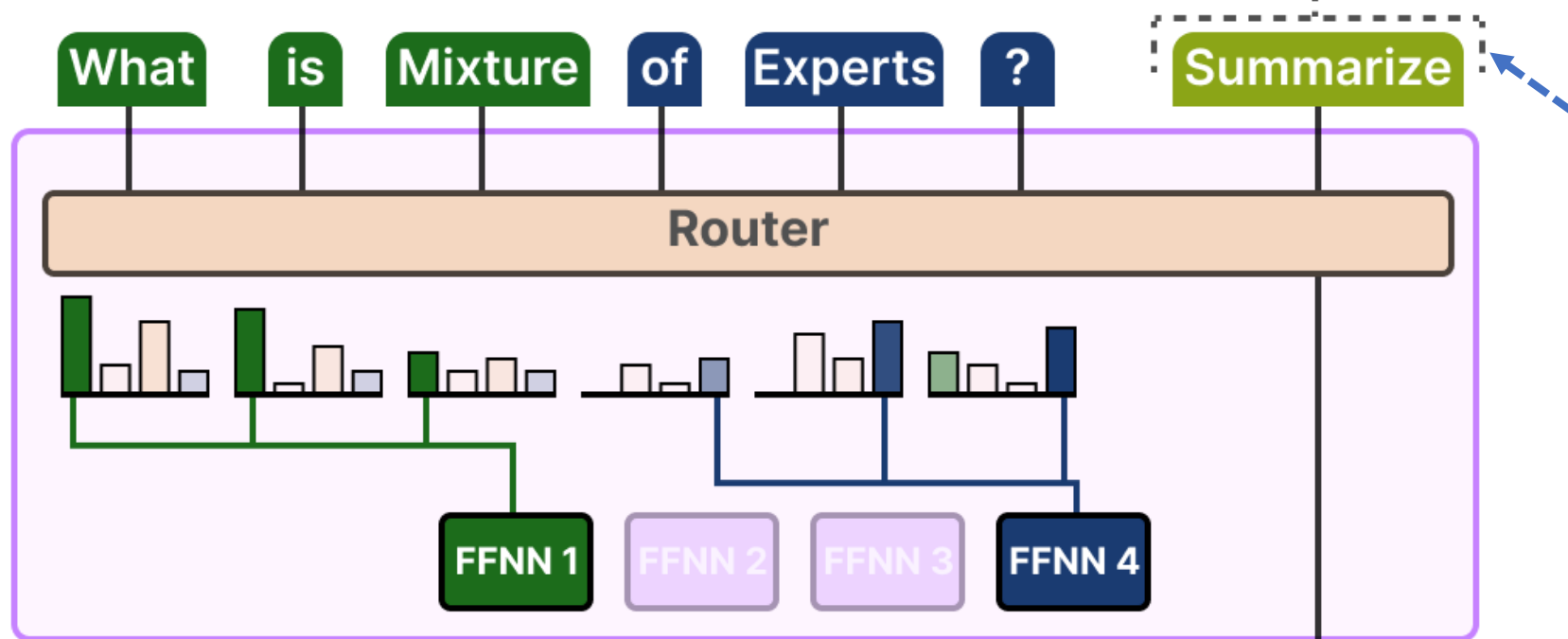
- Expert Capacity
 - S: # of tokens
 - E: # of experts
 - K: Tok-K parameter

Mixture of Experts

- Expert Capacity

- limiting the amount of tokens a given expert can handle

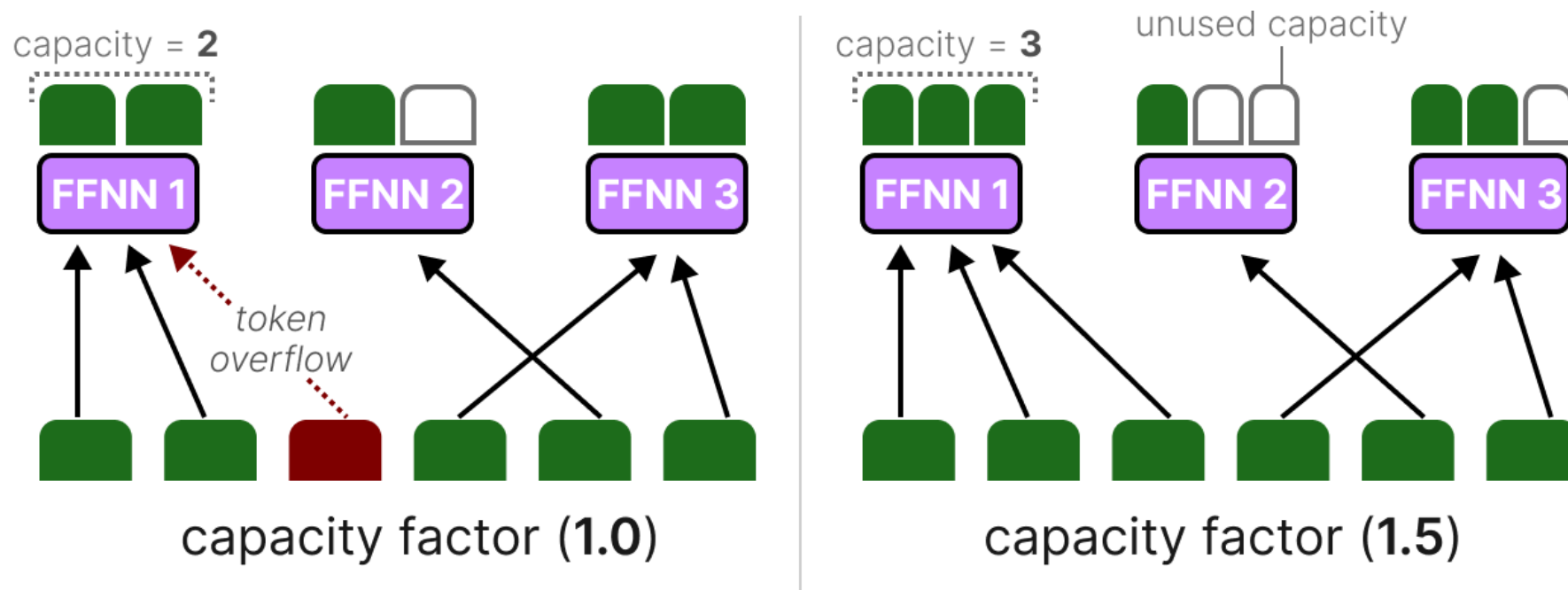
*FFNN 1 and FFNN 4 have **reached capacity**.
The token is sent to the **next layer** instead.*



Overflow: not trained by any expert

Mixture of Experts

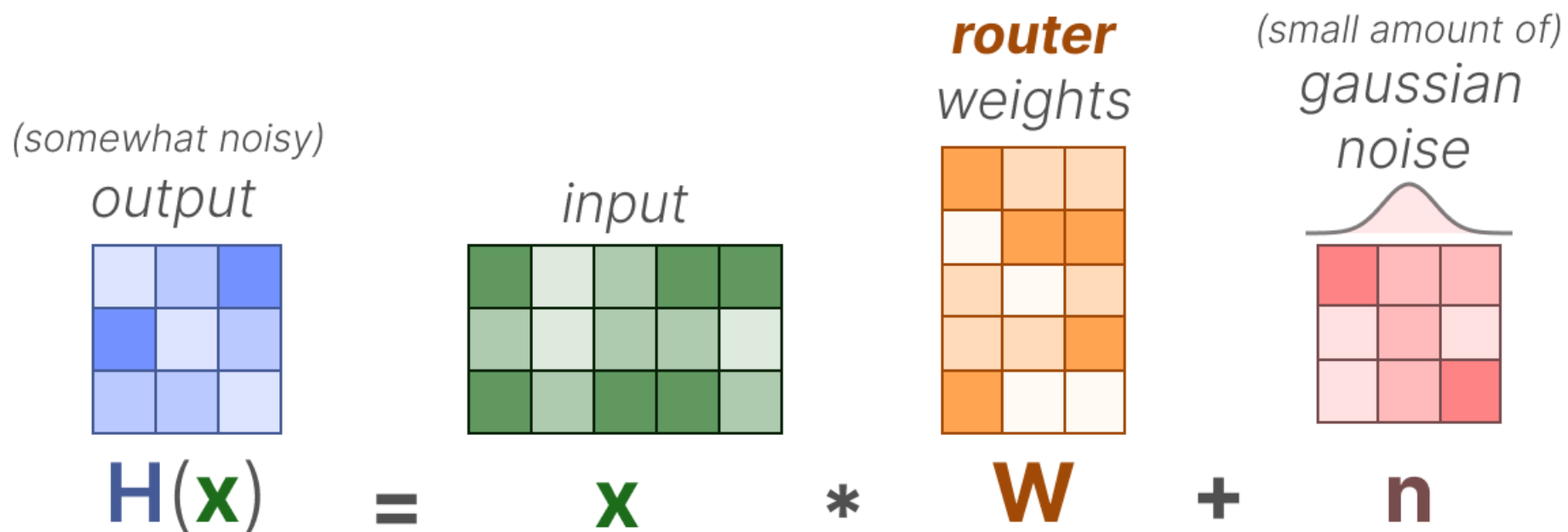
- Adjusting Expert Capacity



Increasing the capacity factor increases the quality but increases communication costs and memory of activations.

Mixture of Experts

- Random Routing/Noisy Routing

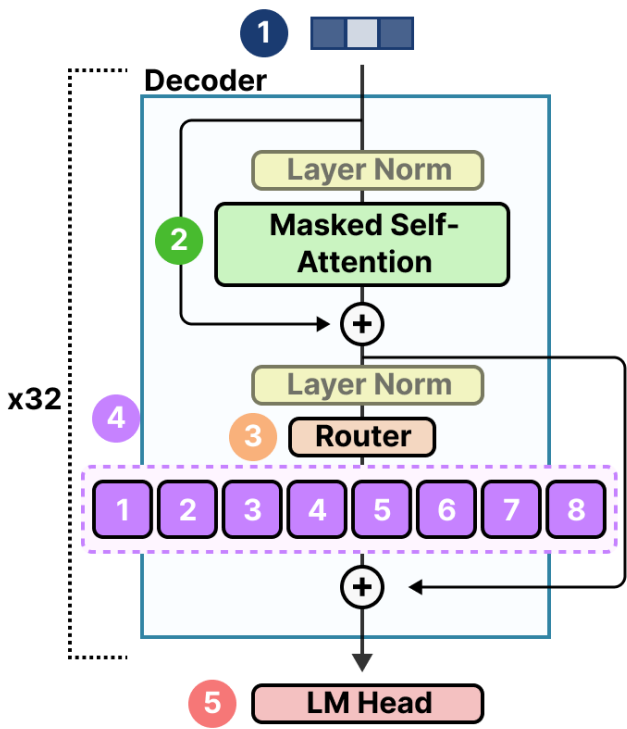


preventing the same experts from always being selected

Mixture of Experts

- Scaling up model parameters

Mixtral 8x7B



1	Embeddings	$32000 \times 4096 = 131.072.000$	shared parameters
2	Attention	$32 \times 41.943.040 = 1.342.177.280$	shared parameters
3	Router	$8 \times 4096 = 32.768$	shared parameters
4	Experts	$8 \times 5.637.144.576 = 45.097.156.608$	total parameters
		$2 \times 5.637.144.576 = 11.274.289.152$	active parameters
5	LM Head	$32000 \times 4096 = 131.072.000$	shared parameters

Shared parameters					
1	Embeddings	131.072.000	1	Embeddings	131.072.000
2	Attention	1.342.177.280	2	Attention	1.342.177.280
3	Router	32.768	3	Router	32.768
4	Experts	45.097.156.608	4	Experts	11.274.289.152
5	LM Head	131.072.000	5	LM Head	131.072.000
Sparse Parameters (all parameters)		46.7B	Active Parameters (activated parameters)		12.8B

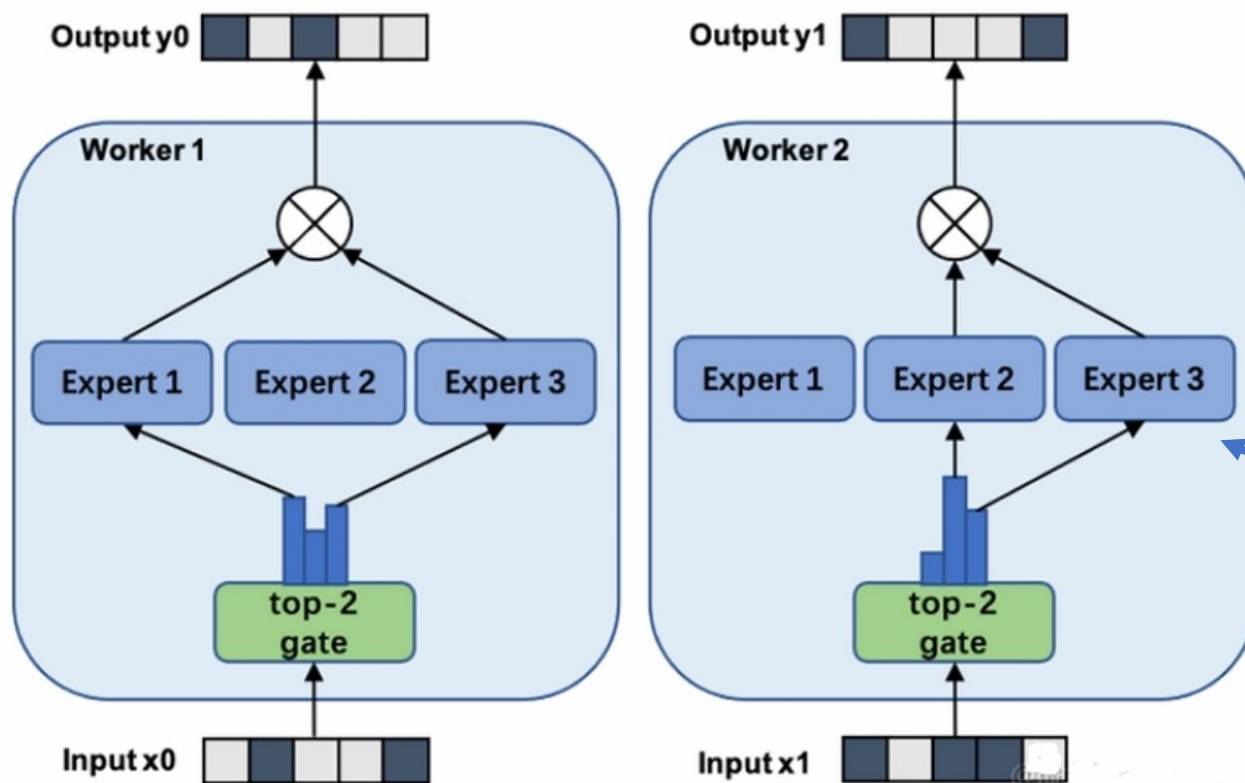
Significant Reduction on Computations

Distributed LLM Training: Outline

- Data Parallelism
- Model Parallelism
- Mixture of Experts
 - Principles
 - **Parallel Training**
 - Recent Progresses

Mixture of Experts

- Parallel Training
 - How to distribute experts on different GPUs?



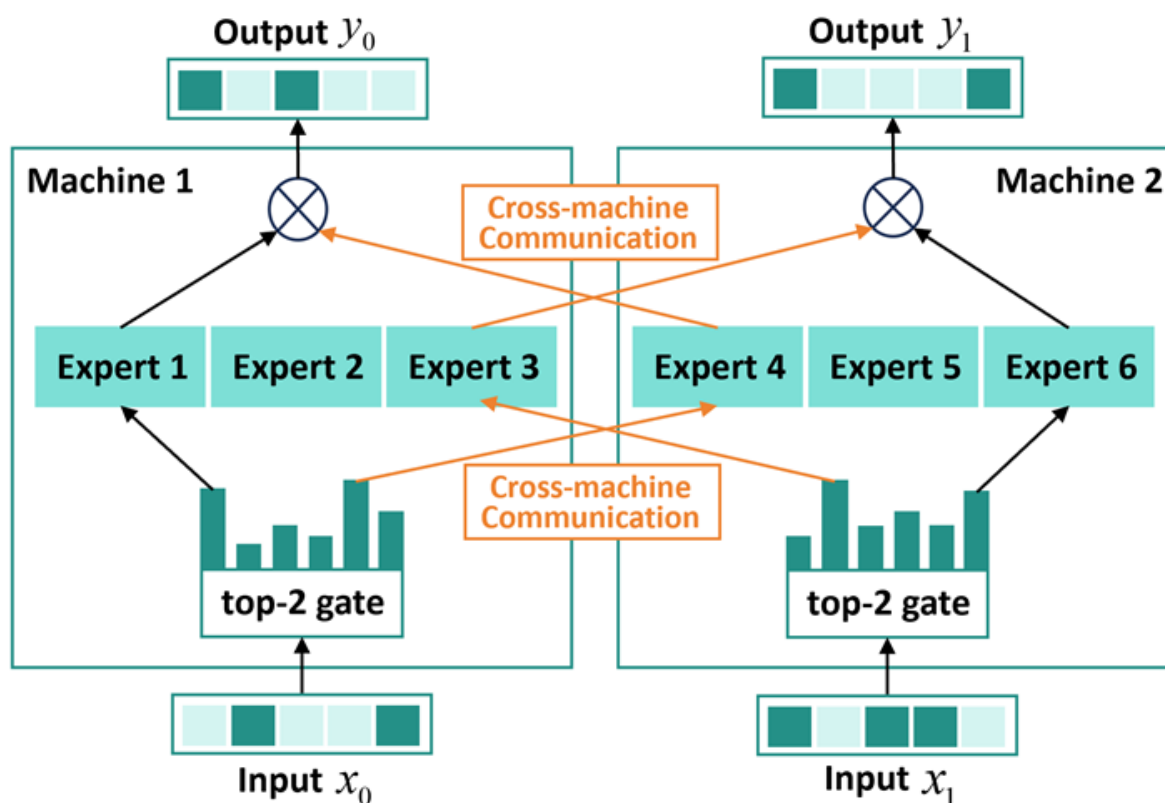
- Data and Expert Parallelism
 - Small # of experts
 - Experts are small in sizes

Every GPU/Machine stores the complete replicas of experts

Mixture of Experts

- Parallel Training

- How to distribute experts on different GPUs?

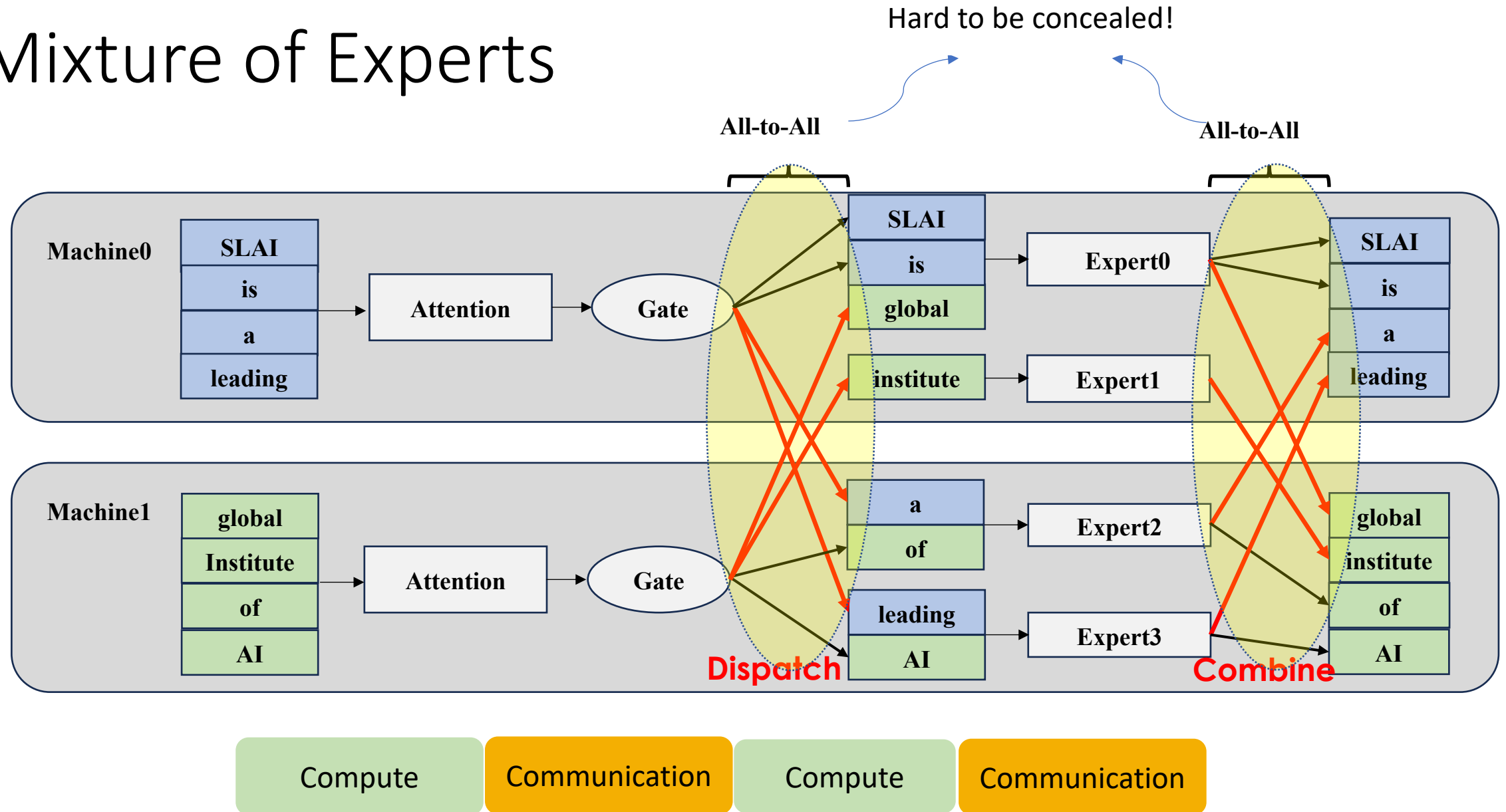


6 experts, DP = 2, EP = 2 (assuming one machine one GPU)

- Data and Expert Parallelism
 - Scaling up to gigantic experts
 - Experts are placed across DP groups
- Size of a “token”
 - $hidden_dim * 2$ Bytes (fp16)
 - GPT3: 7168 ($hidden_dim$)
 - GPT4: 16384 ($hidden_dim$)

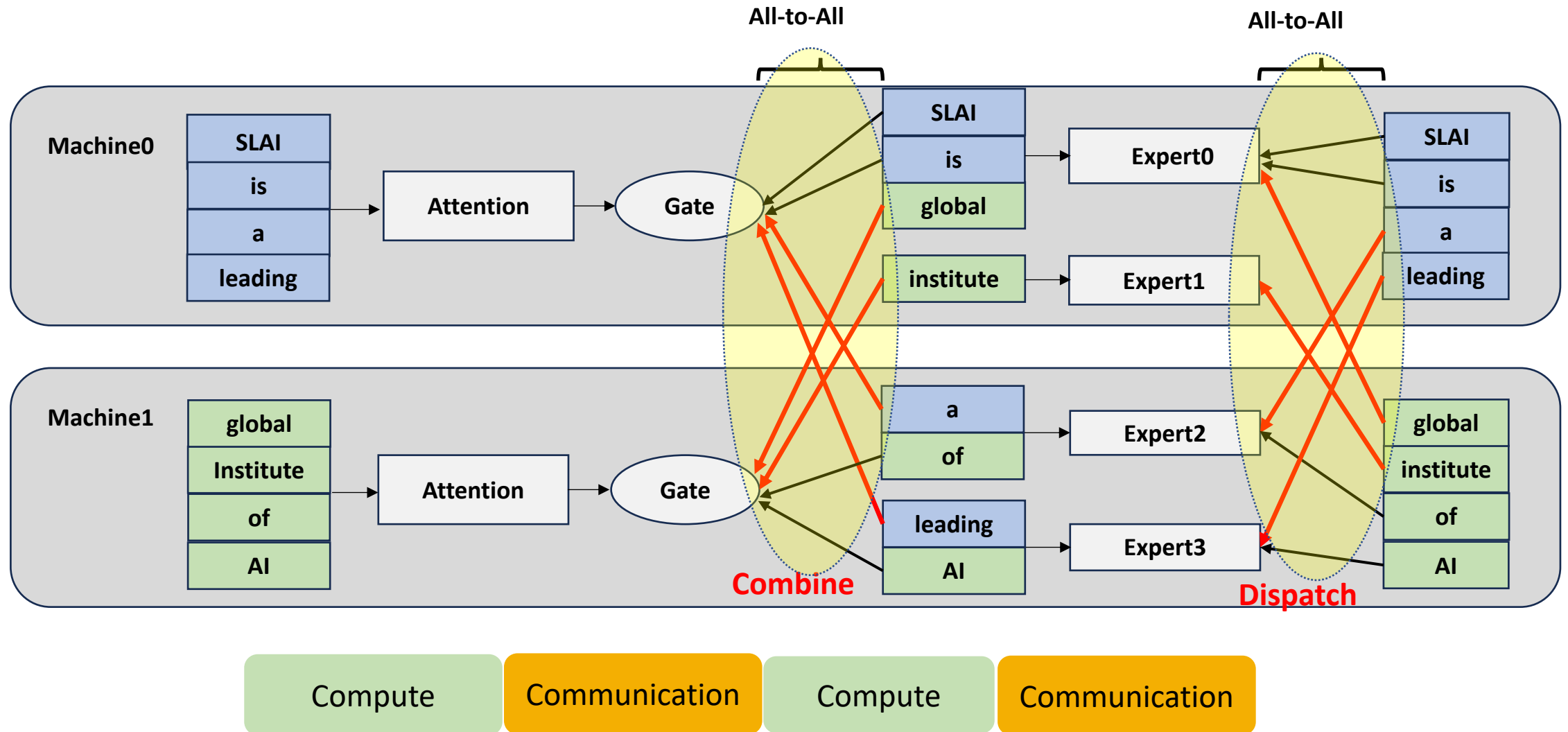
Cross-Machine Communication

Mixture of Experts

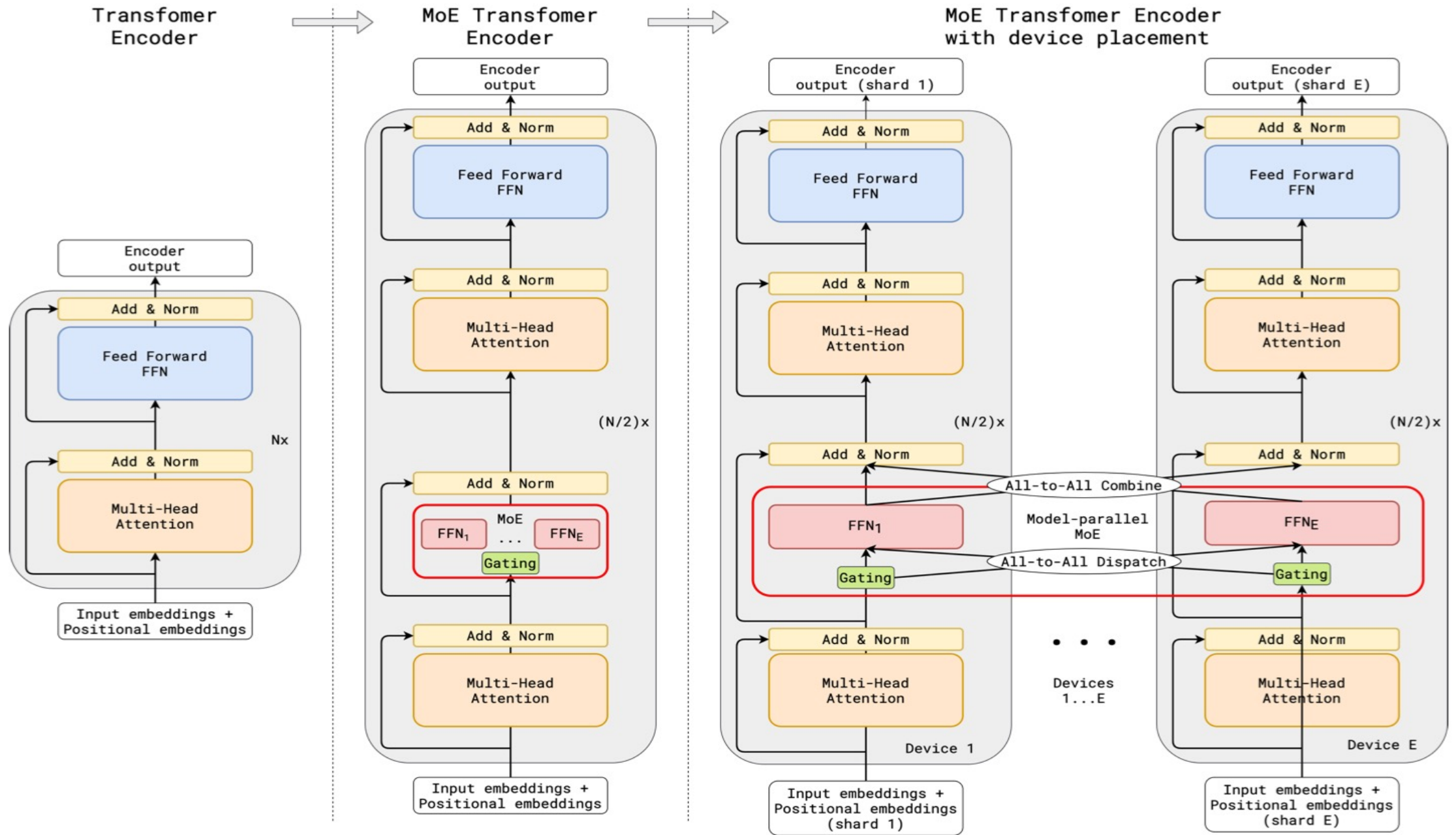


Forward Pass: heavy token communications that hard to be concealed

Mixture of Experts

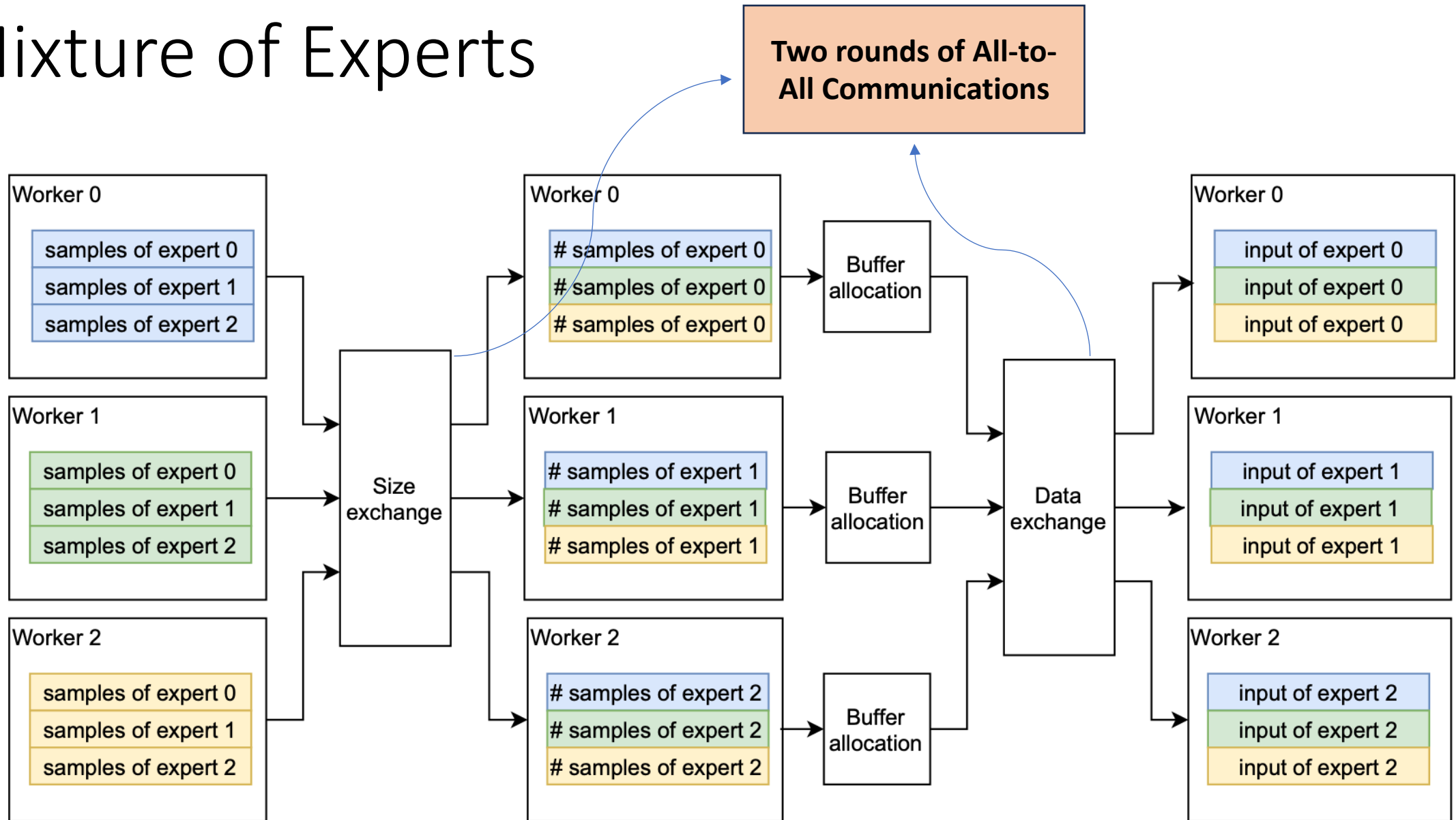


Backward Pass: heavy token communications that hard to be concealed



GShard (2020): imaging the workflow of forward and backward propagations

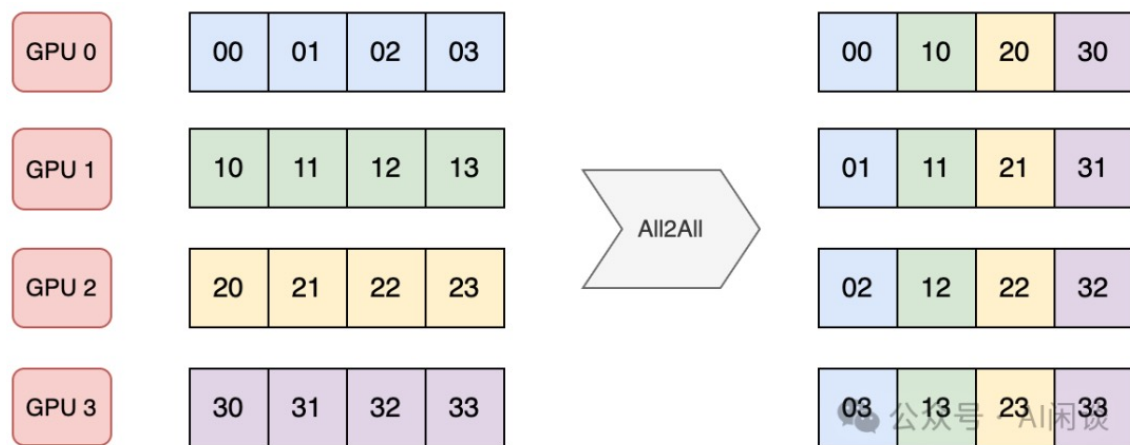
Mixture of Experts



FastMoE (2021): First MoE training system that supports Pytorch

Mixture of Experts

- All-to-All Communication
 - Standard all-to-all (symmetric)



```
import torch
import torch.distributed as dist

def run_alltoall():
    rank = dist.get_rank()
    size = dist.get_world_size()
    device = torch.device(f'cuda:{rank}')
    input_tensor = torch.ones(size, dtype=torch.int32, device=device) * rank
    print(f"Rank {rank} before alltoall, input_tensor: {input_tensor.tolist()}")

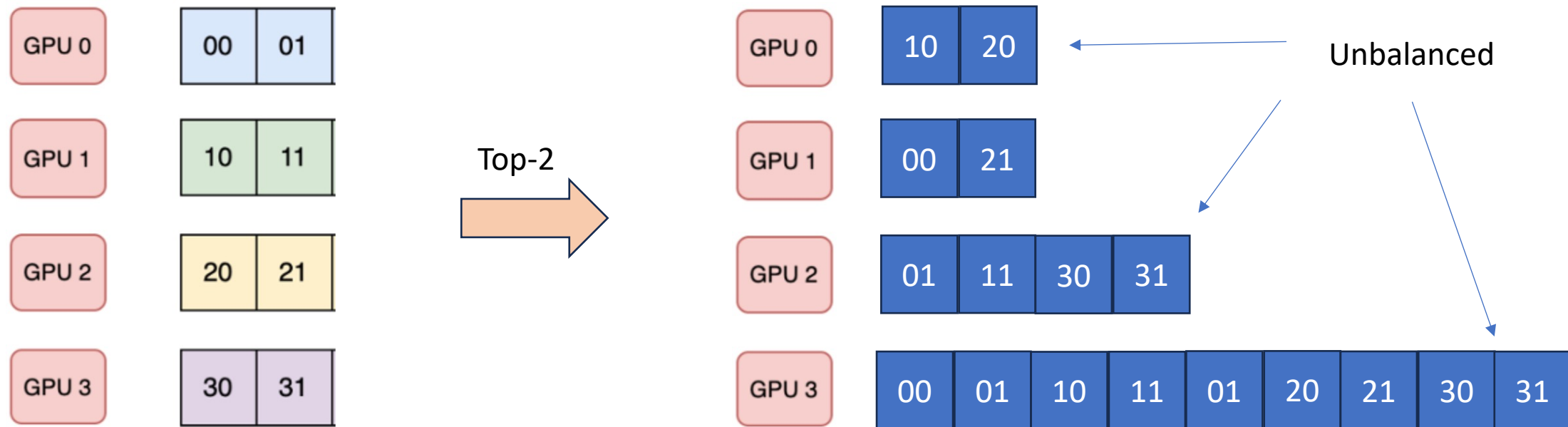
    output_tensor = torch.empty(size, dtype=torch.int32, device=device)
    dist.all_to_all_single(output_tensor, input_tensor)
    print(f"Rank {rank} after alltoall, output_tensor: {output_tensor.tolist()}")

def main():
    dist.init_process_group(backend='nccl')
    run_alltoall()
    dist.destroy_process_group()

if __name__ == "__main__":
    main()
```

Mixture of Experts

- All-to-All Communication
 - Asymmetric data volume (tokens) between GPUs hosting experts
 - **ZERO padding** to transmit *dummy* tokens under standard All-to-All?



Mixture of Experts

- All-to-All Communication
 - NCCL (*NVIDIA Collective Commun. Library*) does not have all-to-all implementations
 - ***ncclSend*** and ***ncclRecv*** **point-to-point** communication instead

```
# 伪代码表示NCCL的all-to-all实现
ncclGroupStart()
for target_rank in range(world_size):
    if target_rank != my_rank:
        ncclSend(send_buffer[target_rank], target_rank, comm)
        ncclRecv(recv_buffer[target_rank], target_rank, comm)
ncclGroupEnd()
```

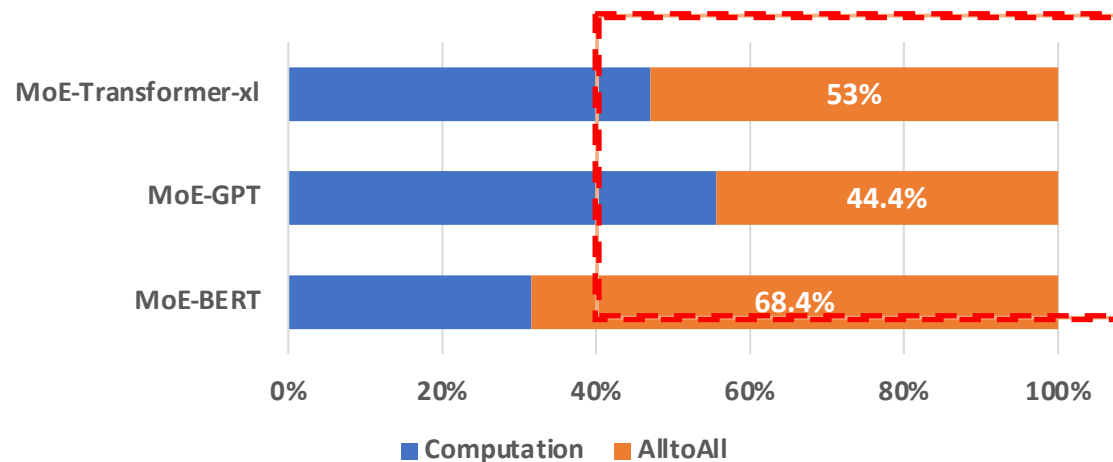
Pseudo codes for “spread-out” implementation

Mixture of Experts

- MoE All-to-All Communication Practice
 - Dispatch
 - **Gathering # of tokens to send and receive**: one standard All-to-All to exchange the # of tokens to transfer, e.g. via ***torch.distributed.all_to_all_single*** in DeepSpeed-MoE
 - Creating sending and receiving buffers, e.g. by ***appending*** or ***concatenating*** vectors
 - Exchanging token vectors: non-standard All-to-All, e.g. via ***torch.distributed.all_to_all_single*** while the # of tokens differ
 - Assigning received tokens to local experts
 - Combine
 - Creating sending and receiving buffers
 - Exchanging the expert outputs and meta data to the corresponding nodes

Mixture of Experts

- Challenge in MoE All-to-All



All-to-All communication accounts for a significant proportion (44.4% ~ 68.4%),

iteration time vs time of All-to-All (32 GPUs)

	#Parameter	#layer	#AlltoAll per iteration
GPT* + MoE	175 B	96	96 * 4 = 384 times

Upscaling with # of transformer blocks
(an example)

Distributed LLM Training: Outline

- Data Parallelism
- Model Parallelism
- Mixture of Experts
 - Principles
 - Parallel Training
 - **Recent Progresses**

Mixture of Experts

- Optimizing MoE All-to-All Communication
 - Expert-centric: keeping **experts in-place** and moving tokens via All-to-All
 - Data-centric: keeping **tokens in-place** and moving experts between GPUs

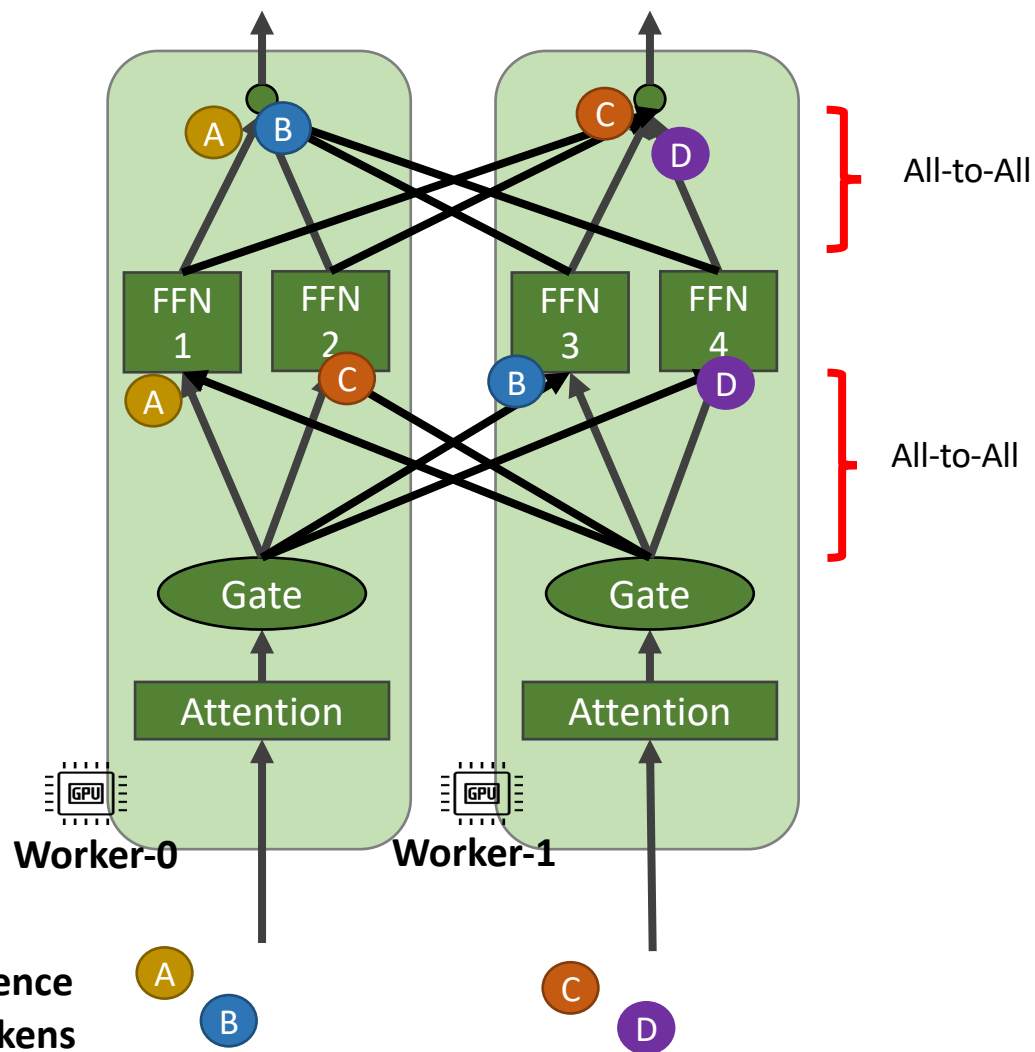
Case Study:

Running on 4 8-A100 machines (32 GPUs)
Batch=64, Seq=512, Gate's topk=2, hidden size=256, #expert=32

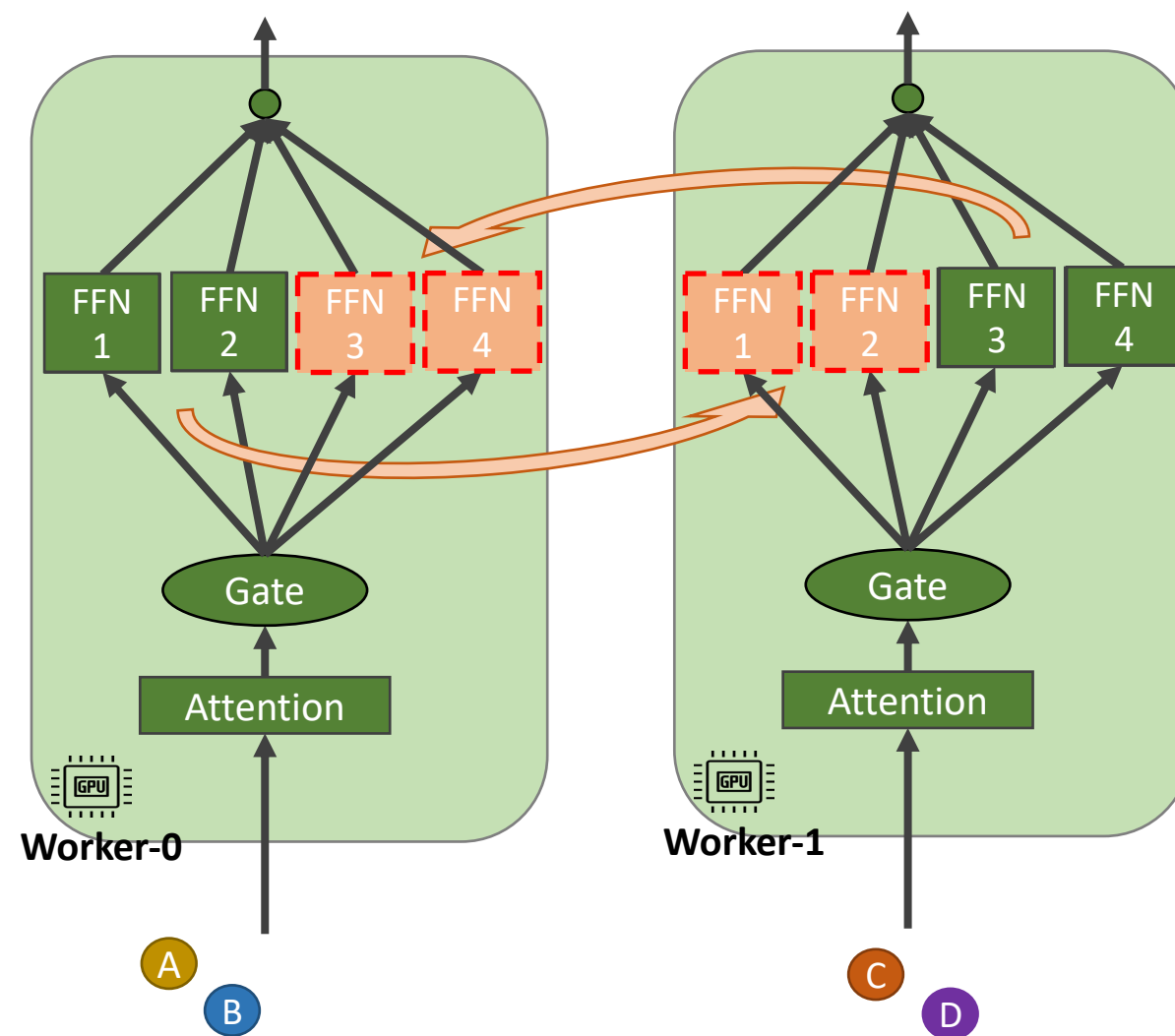
	Operation	Volume of Traffic across a machine
Expert Centric	send out tokens	0.75GB
Data Centric	fetch experts	0.047GB

Data Centric has lower traffic workload !

Expert Centric

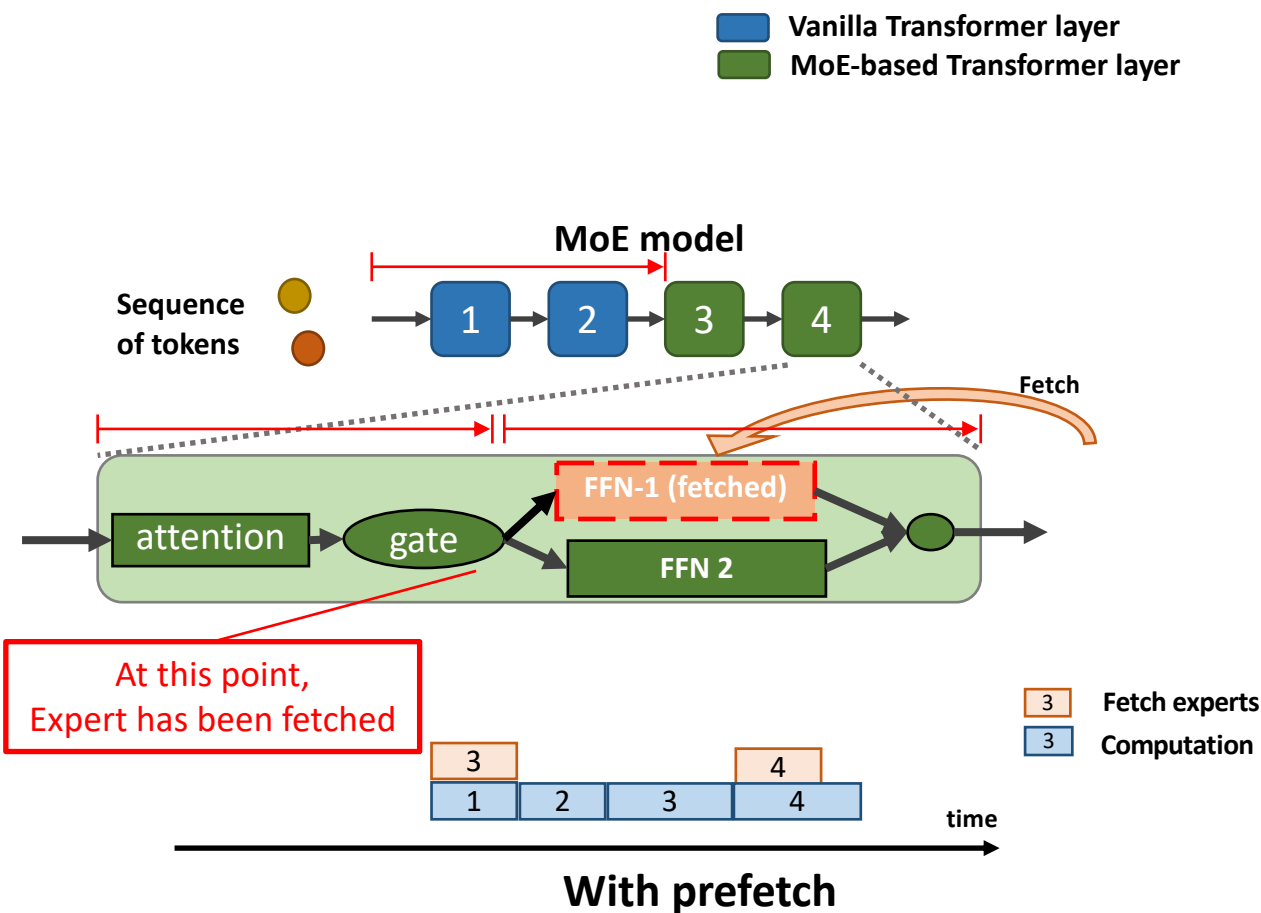
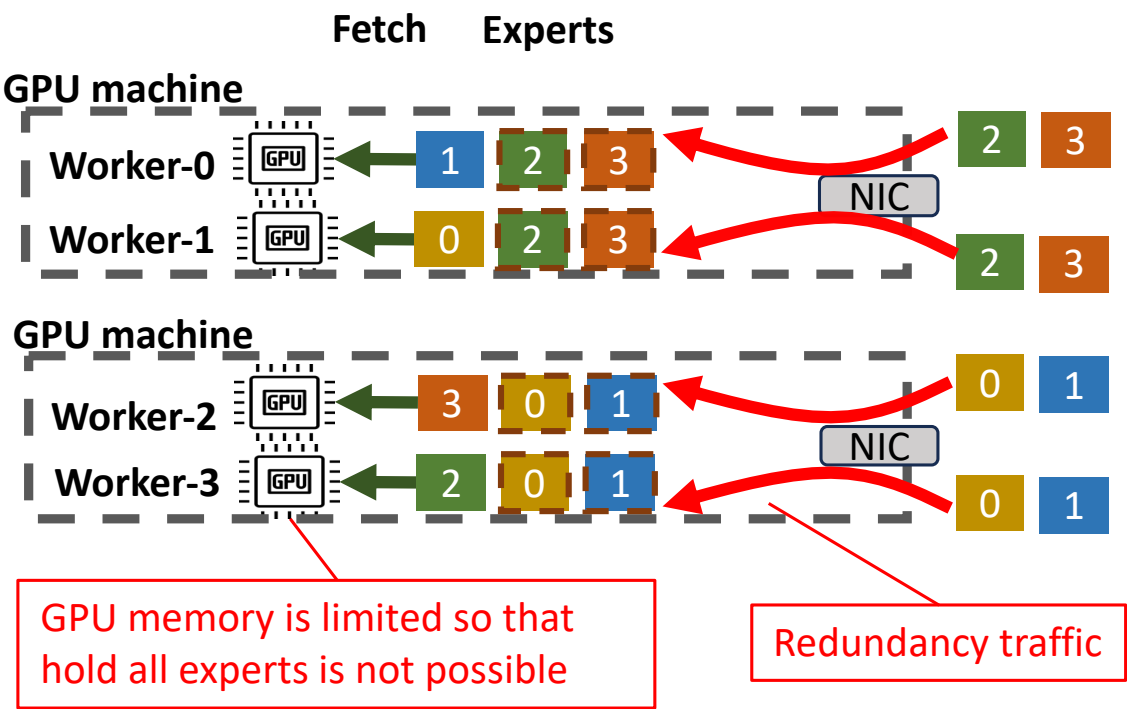


Data Centric



Mixture of Experts

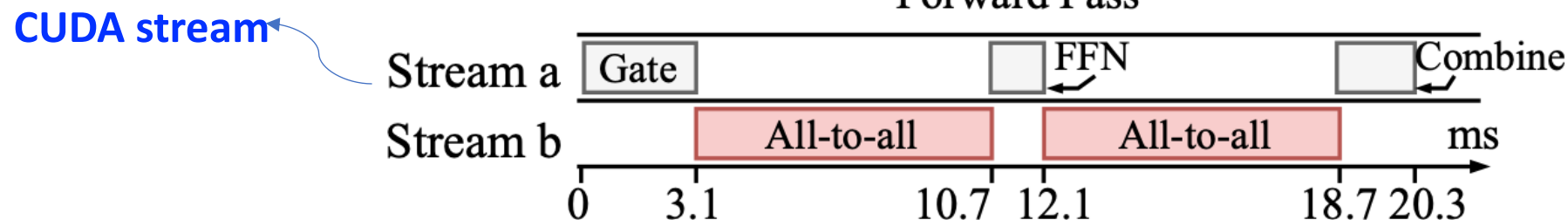
- Optimizing MoE All-to-All Communication
 - Potential challenges in Janus



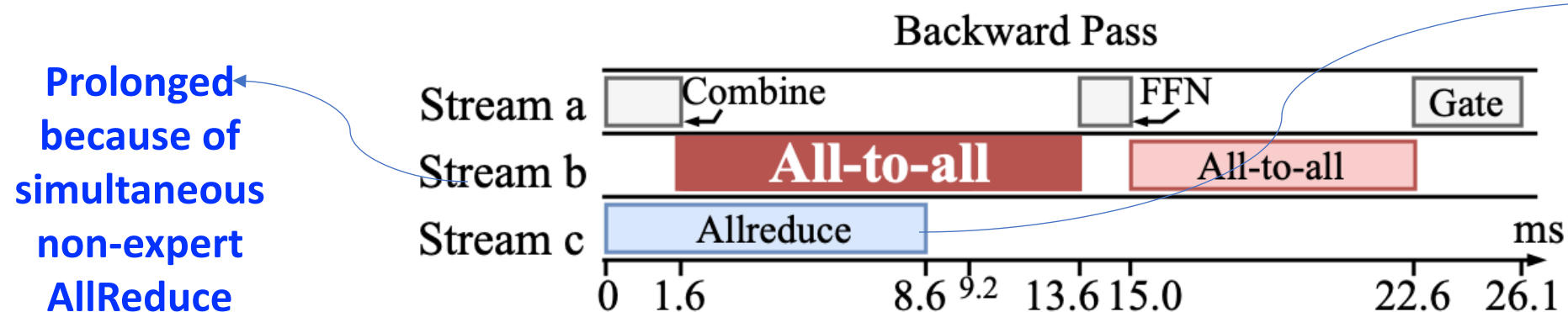
Mixture of Experts

- Optimizing MoE All-to-All Communication
 - Lina (2023): forward pass is synchronous and blocking

Forward Pass



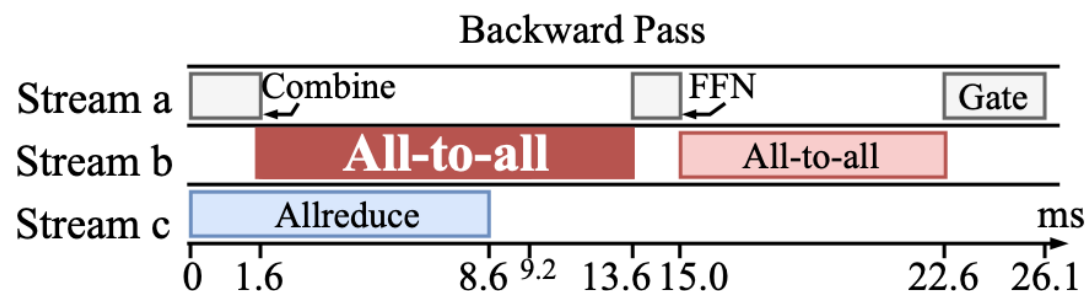
- Lina: backward pass is



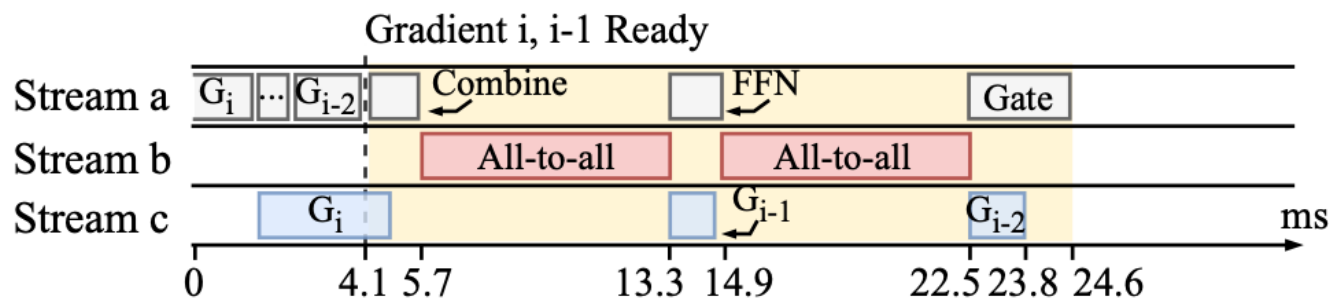
asynchronously
aggregate non-
expert gradients
in data parallel

Mixture of Experts

- Optimizing MoE All-to-All Communication
 - Lina: separate CUDA streams for the expert-parallel and data-parallel groups



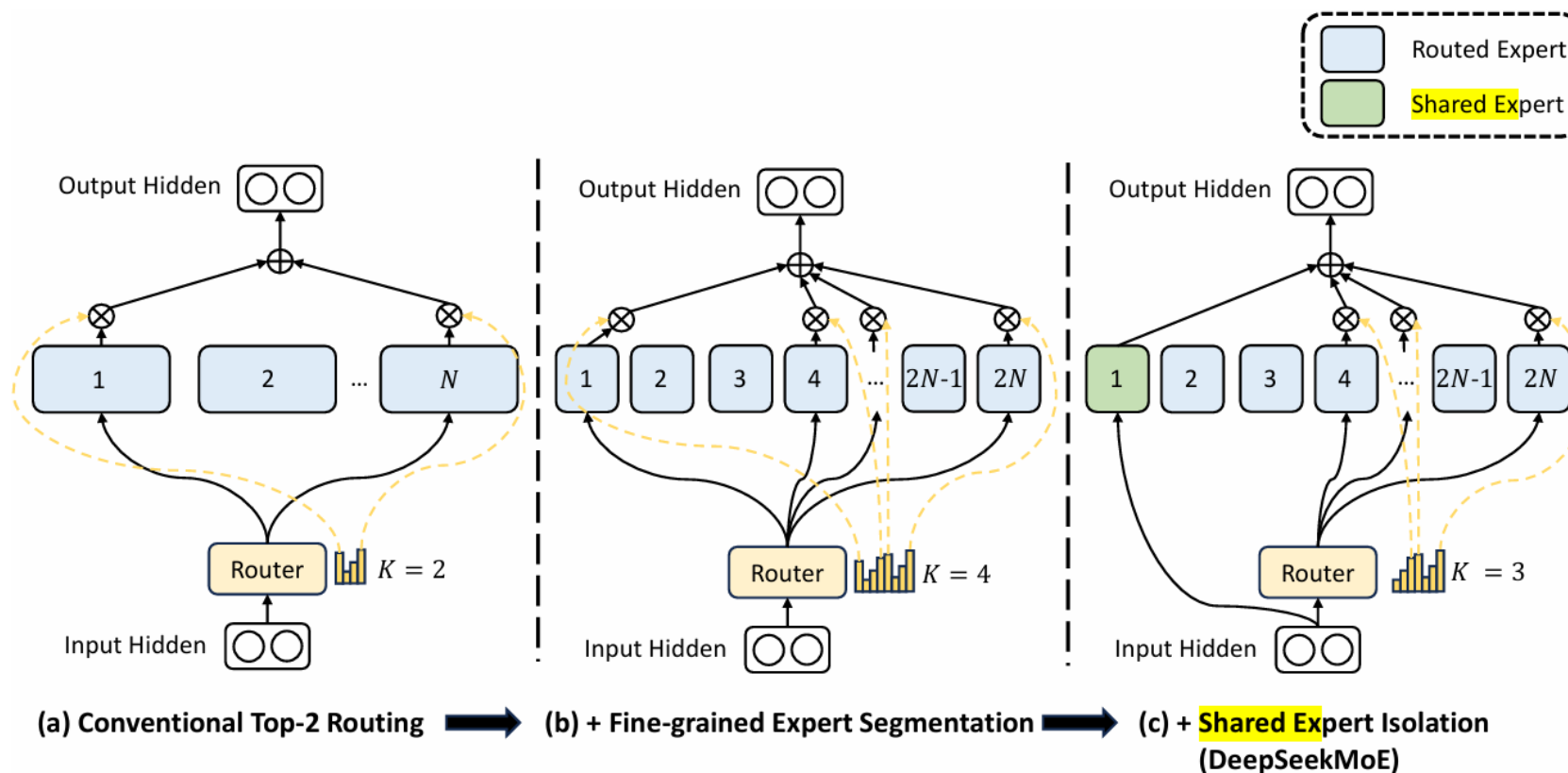
- Lina: always prioritize all-to-all and avoid bandwidth sharing



- More recent approaches: FSMoE (2025), ScheMoE (2024), etc.

Mixture of Experts

- Overlapping MoE All-to-All in DeepSeek v3



DeepSeek v3: 61 layers, 671B parameters, **256 routing experts**, **Top-8 selection**, **37B parameters** activated by one token

Mixture of Experts

- Overlapping MoE All-to-All in DeepSeek v3

Token-expert affinity: $s_{i,t} = \text{Sigmoid}(u_t^T e_i)$



Token-expert affinity: $s_{i,t} = \text{Sigmoid}(u_t^T e_i) + b_i$

Adjusting b_i dynamically

Token-level load balancing

The Shenzhen Loop Area Institute (SLAI) is a premier global institution dedicated to artificial intelligence, serving as a national pilot for innovative AI talent cultivation under the auspices of China's Ministry of Education.

Sequence-level load balancing

Each token can be sent to at most 4 machines

Cross-machine restriction

Mixture of Experts

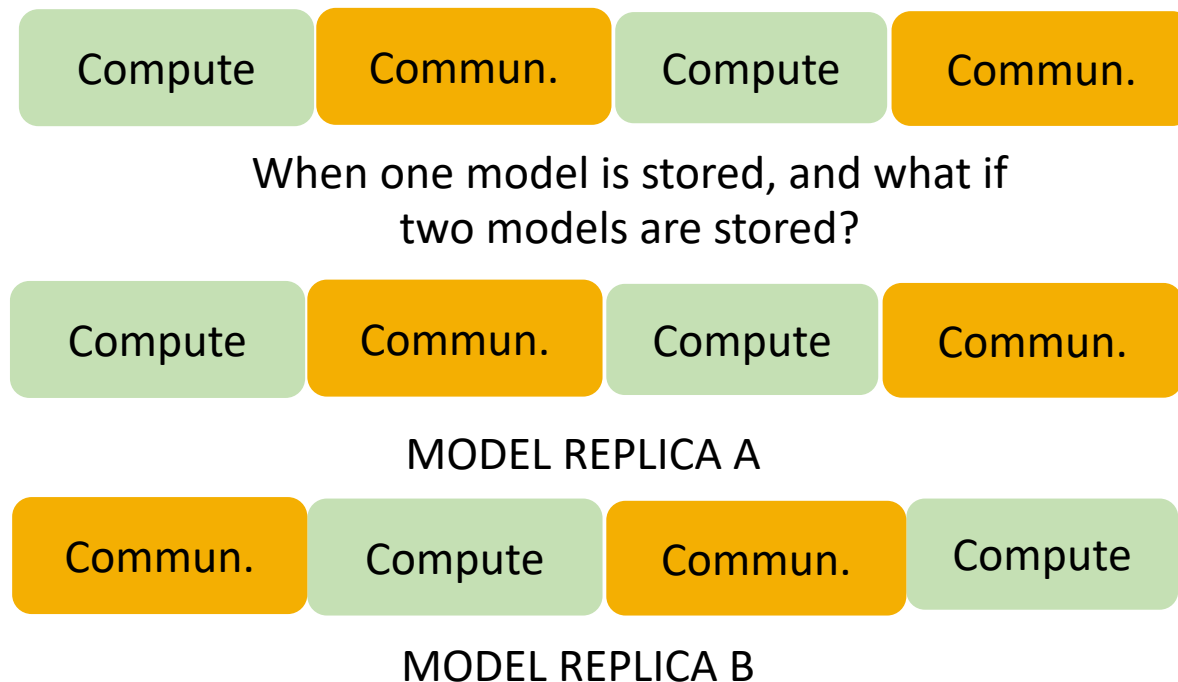
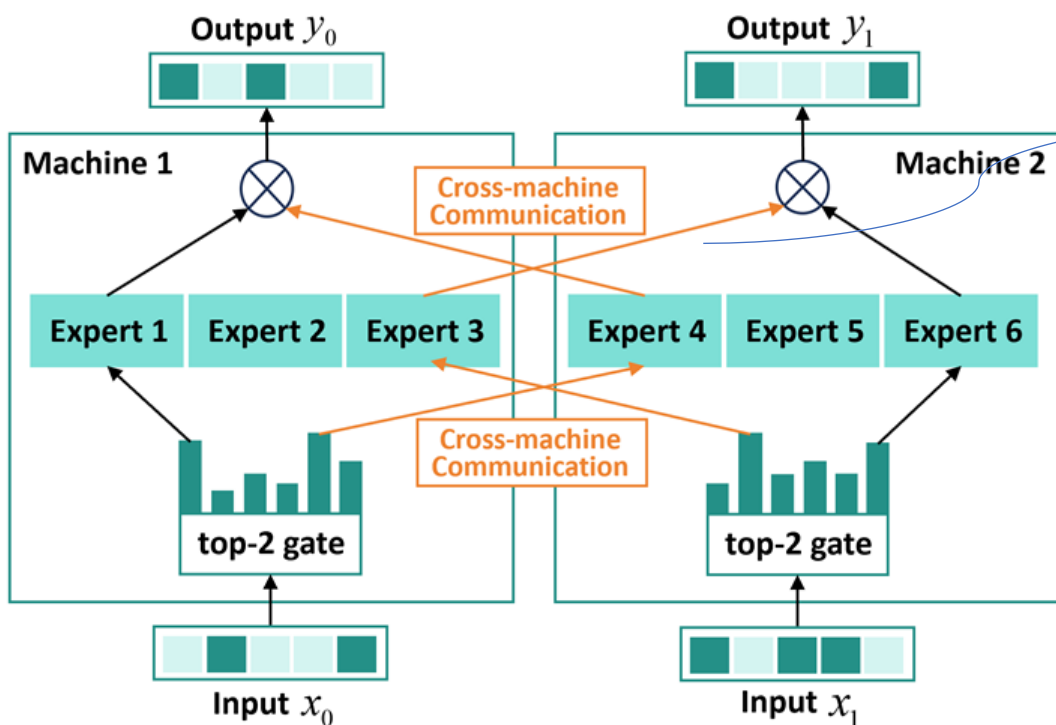
- Overlapping MoE All-to-All in DeepSeek v3
 - 1F1B pipeline: forward pass followed by the immediate backward pass
 - **ZERO bubble**: decomposing backward into backwards for input and parameter
 - **Chimera**: bidirectional pipeline for squeezing bubbles



DualPipe scheduling for 8 PP ranks and 20 micro-batches in two directions. The micro-batches in the reverse direction are symmetric to those in the forward direction

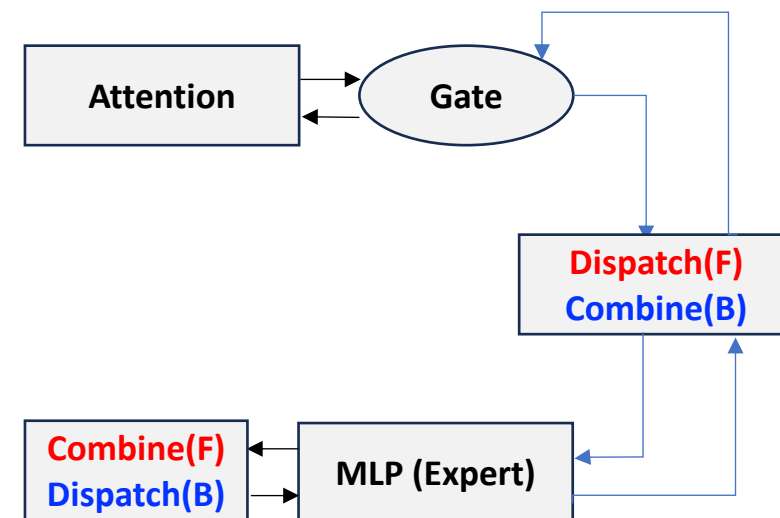
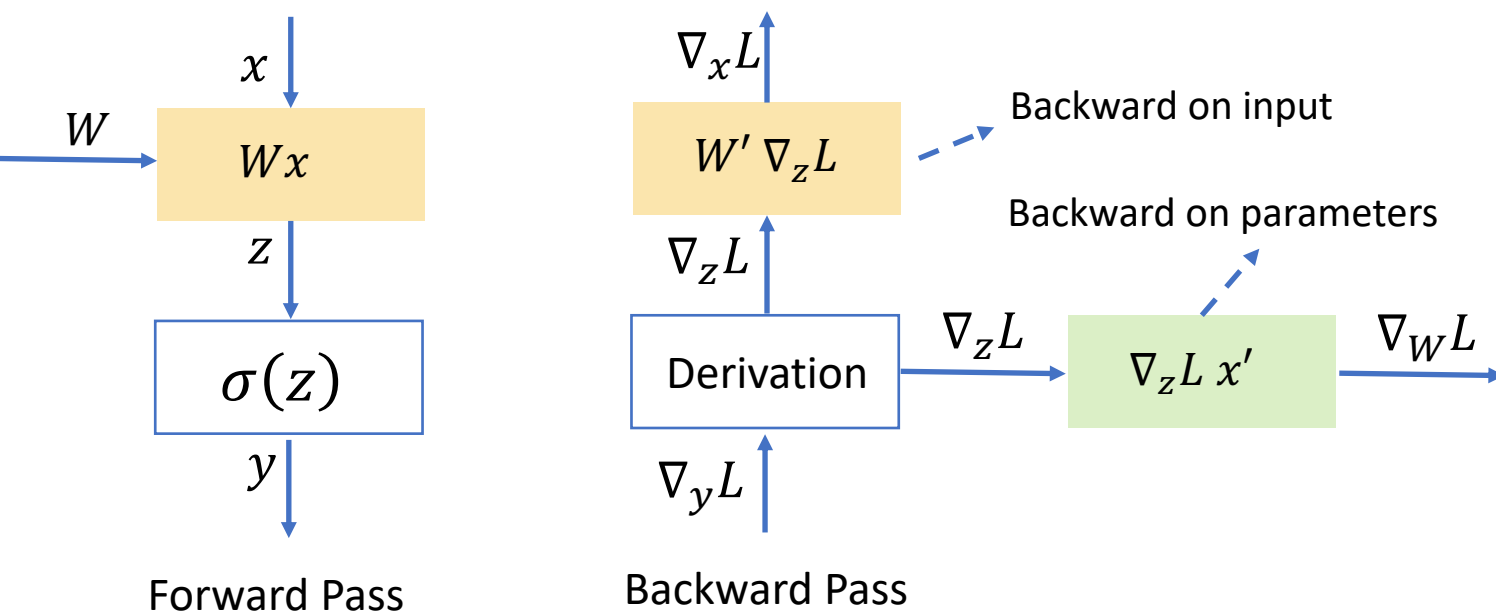
Mixture of Experts

- Overlapping MoE All-to-All in DeepSeek v3
 - Why All-to-All inter-machine communication can hardly be hidden?
 - Dependencies between compute and communication need to be decoupled!



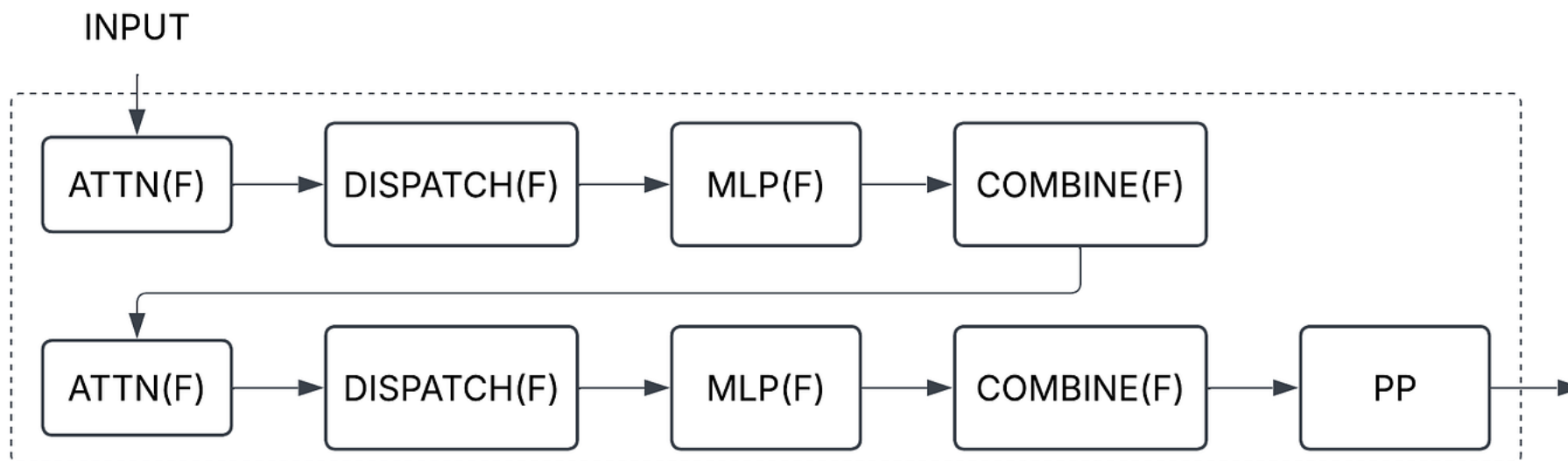
Mixture of Experts

- Overlapping MoE All-to-All in DeepSeek v3
 - Decomposing computations into *ATTN(F)*, *MLP(F)*, *ATTN(BI)*, *ATTN(BW)*, *MLP(BI)*, *MLP(BW)*
 - Differentiating *DISPATCH(F)*, *DISPATCH(B)*, *COMBINE(F)*, *COMBINE(B)*
 - *DISPATCH(B)* after *MLP(BI)*, *COMBINE(B)* after *ATTN(BI)*



Mixture of Experts

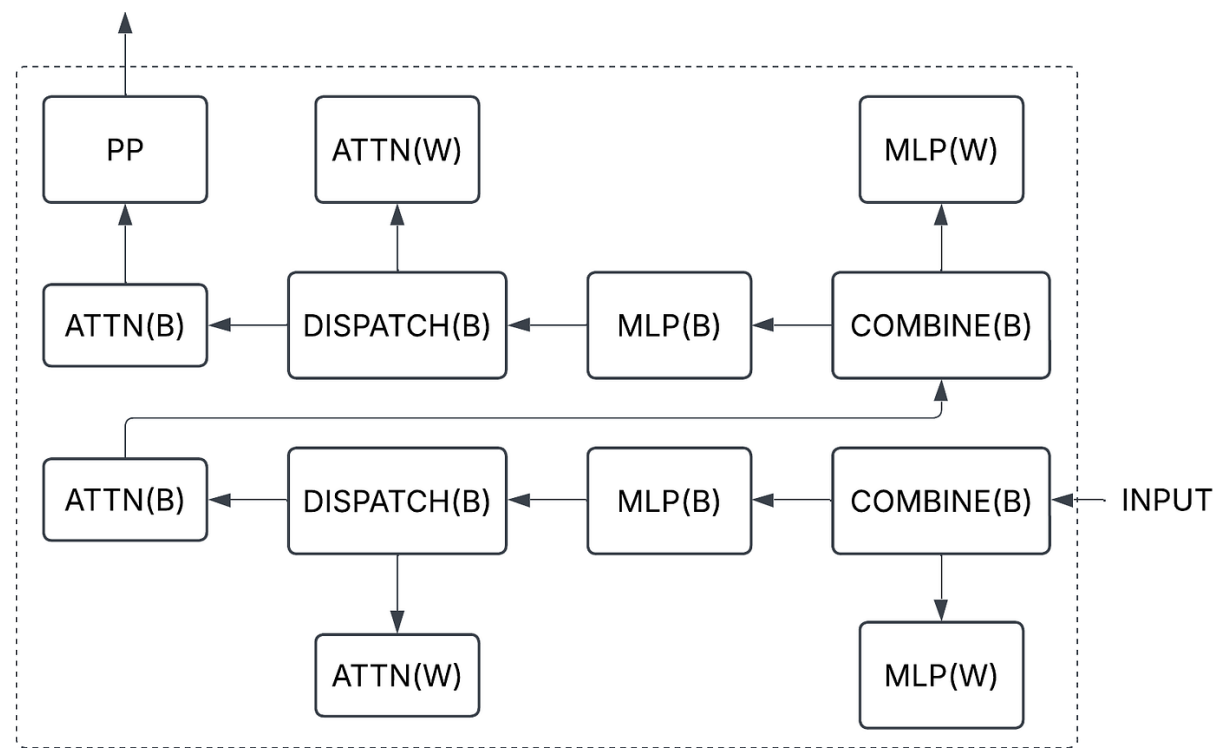
- Overlapping MoE All-to-All in DeepSeek v3
 - Forward pass



Decouple compute and communication as much as possible

Mixture of Experts

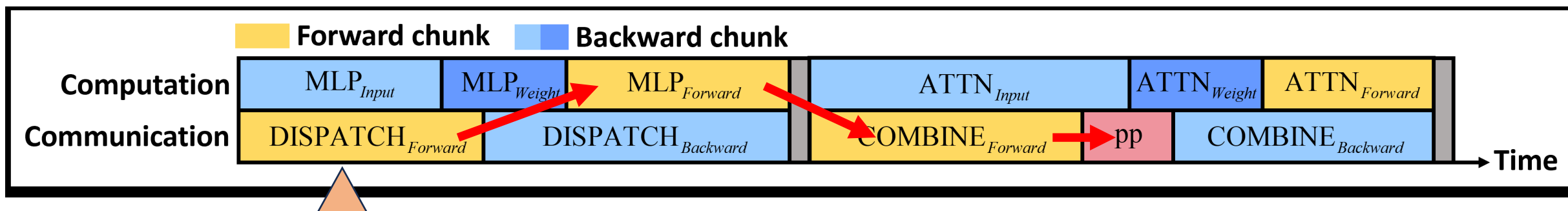
- Overlapping MoE All-to-All in DeepSeek v3
 - Backward pass



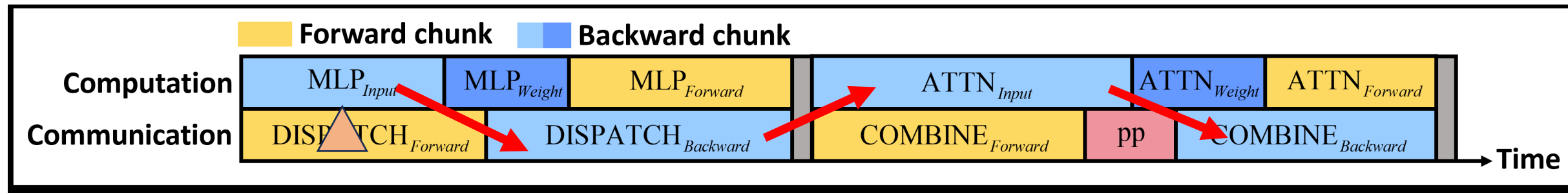
Decouple compute and communication as much as possible

Mixture of Experts

- Overlapping MoE All-to-All in DeepSeek v3
 - Forward pass

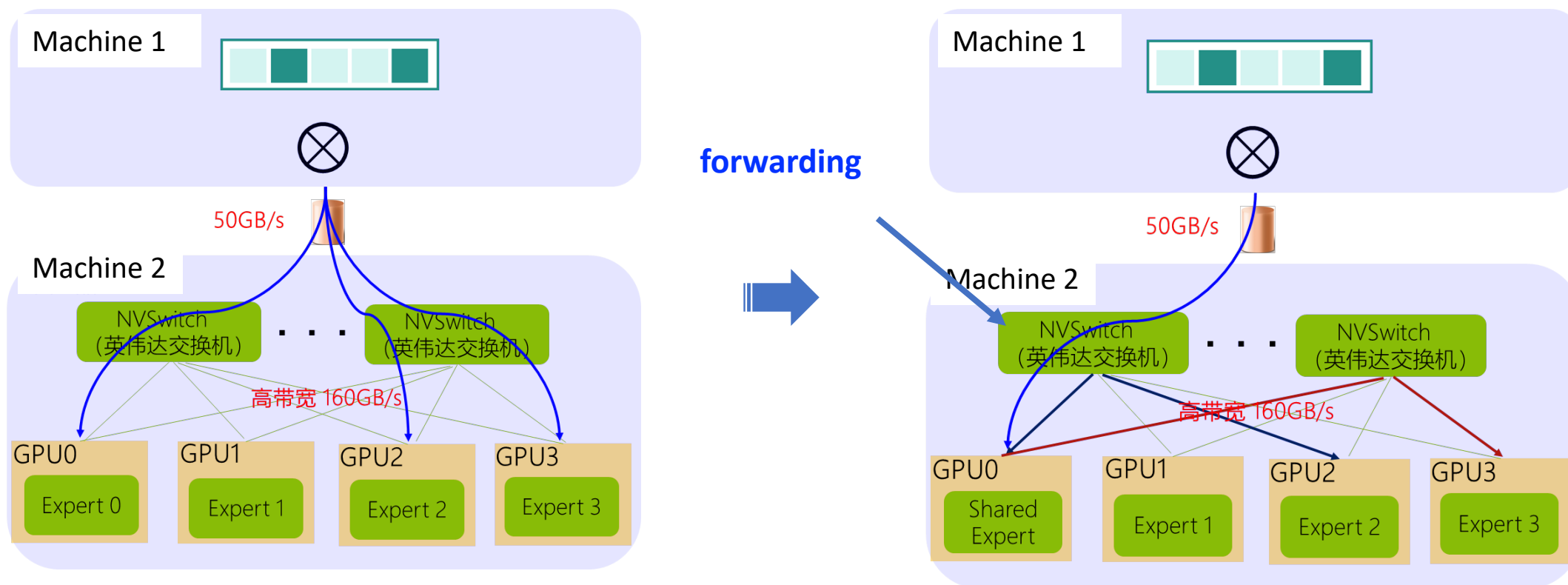


- Backward pass



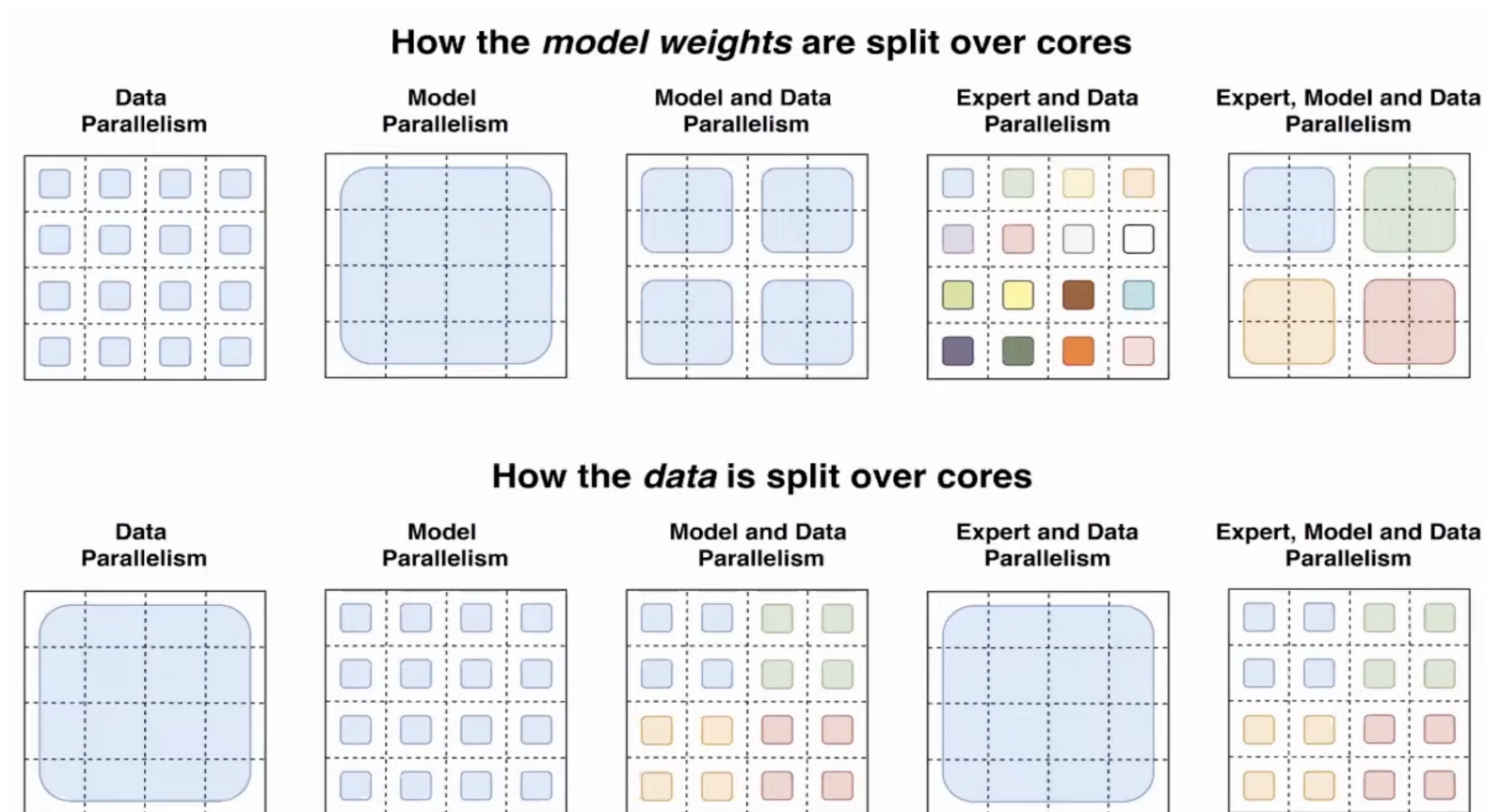
Mixture of Experts

- Overlapping MoE All-to-All in DeepSeek v3
 - Token transfer: forwarding from RDMA to NVLink by use of asymmetric bandwidth



Hybrid Parallelism

- Segmenting data, model and activation



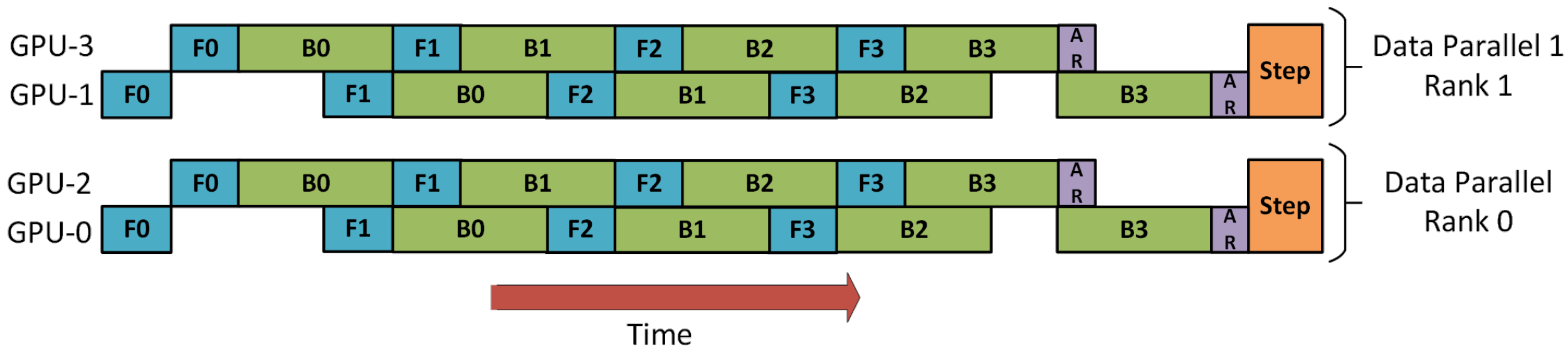
A Brief Summary

- Hybrid parallelism (classical models)

Models	TP	DP	PP	Micro batchsize	global batchsize	sequence length	ZeRO	GPUs	GPU types
Bloom-176B	4	8	12	2	2048	2048	ZeRO-1	384	A100-80GB
Megatron-175B	8	\	16	1	1536	2048	\	\	\
OPT-175B	8	124	\	\	\	2048	\	996	A100-80GB
GPT3-175B	\	\	\	\	\	2048	\	\	\
Megatron-NLG 530B	8	16	35	\	1960	2048	\	4480	A100-80GB
GLM-130B	4	24	8	\	\	4096	ZeRO-1	768	A100-80GB

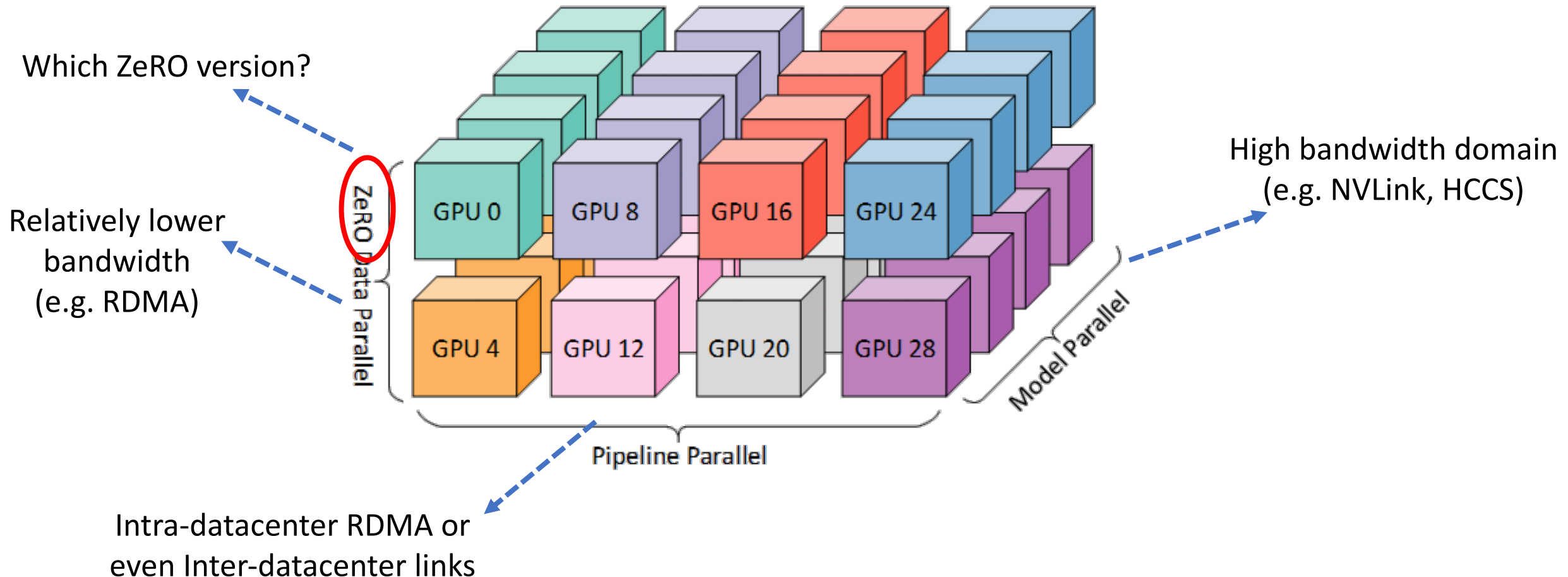
A Brief Summary

- Data Parallelism + Pipeline Parallelism



A Brief Summary

- Data Parallelism + Pipeline Parallelism + Tensor Parallelism + ZeRO



Thanks!

